

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.M71-MM

Подсистема CRC Python IDE для анализа и ассистирования модификаций кода

Кузнецов Илья Александрович

Отчёт по преддипломной практике
в форме «Производственное задание»

Научный руководитель:
профессор кафедры системного программирования,
д. т. н. Кознов Д. В.

Консультант:
ведущий инженер ключевых проектов ООО «Техкомпания Хуавэй»,
Тропин Н. В.

Санкт-Петербург
2026

Оглавление

1. Введение	5
2. Постановка цели и задач	7
3. Обзор предметной области	8
3.1. Статические анализаторы	8
3.2. Среды разработки	9
3.3. Системы на основе ИИ-агентов	11
3.4. Сравнение инструментов	13
4. Архитектура	17
4.1. Инструменты подсистемы	17
4.2. Улучшения модели кода	21
4.3. Структура подсистемы	26
5. Реализация	30
5.1. Инструменты для статического анализа кода	30
5.2. Инструменты для исправления кода	34
5.3. Инструменты для рефакторинга кода	39
5.4. Информационные ассистенты для пользователей	45
5.5. Информационные ассистенты для ИИ-агентов	48
6. Тестирование и апробация	51
6.1. Модульное тестирование	51
6.2. Регрессионное тестирование	53
6.3. Комплексное тестирование	56
6.4. Апробация	58
7. Заключение	60
Список литературы	62
А. Приложение	65
А.1. Инструменты для статического анализа кода	65

A.2. Инструменты для исправления кода	70
A.3. Инструменты для рефакторинга кода	71
A.4. Информационные ассистенты для пользователей	72
A.5. Информационные ассистенты для ИИ-агентов	73

1. Введение

Разработка программного обеспечения на языке Python охватывает широкий круг прикладных задач — от разработки серверных систем и автоматизации до задач анализа данных и создания решений, связанных с искусственным интеллектом. По данным Stack Overflow Developer Survey [22], Python продолжил заметный рост использования и укрепил позицию одного из наиболее востребованных языков программирования, что связывается, в частности, с его важной ролью в задачах ИИ. Расширение области применения Python предъявляет повышенные требования к качеству его инструментальной поддержки: по мере усложнения проектов у разработчиков оказываются востребованы не только сервисы для удобного написания кода, но и средства, позволяющие своевременно обнаруживать проблемы в разрабатываемых программах, интерпретировать причины этих проблем и безопасно вносить изменения в код.

На этом фоне особую значимость приобретают инструменты анализа и ассистирования модификаций кода. Для практической разработки недостаточно одной лишь проверки синтаксической корректности: необходимы средства, способные выявлять ошибки использования ссылок и типов, поддерживать исправления обнаруженных проблем, выполнять эквивалентные преобразования кода и предоставлять контекстную информацию о типах, декларациях, сигнатурах и документации непосредственно в рабочей среде. Это особенно актуально для Python-средств разработки (Python IDEs), в которых динамическая семантика языка затрудняет статический анализ и усложняет сопровождение крупных проектов. Поэтому в современных Python-средствах разработки возрастает роль инструментов, сочетающих проверку и преобразование кода, а также информационное сопровождение пользователя [9].

Дополнительный импульс к развитию таких инструментов дают системы на основе ИИ-агентов. По данным Stack Overflow Developer Survey [22], 84% разработчиков уже используют либо планируют использовать ИИ-инструменты, а 51% применяют их ежедневно. При

этом распространение ИИ-инструментов не снижает требований к качеству IDE, а, напротив, делает их более строгими: если модификация существующего кода осуществляется с участием большой языковой модели, то для получения предсказуемого результата необходимы не только текст программы, но и структурированные сведения о проблемах, декларациях, сигнатурах и типах, а также управляемые операции изменения кода. Исследования взаимодействия человека и ИИ-агентов в IDE [21] показывают, что эффективность таких сценариев зависит от качества встроенного контекста и средств верификации результатов.

CRC Python IDE [5] разрабатывается в лаборатории языковых инструментов Chebyshev Research Center (CRC) и предназначена для внутреннего использования в крупной технологической компании. Для CRC Python IDE актуальна задача построения целостной подсистемы, покрывающей ключевые сценарии работы с Python-кодом. Для этого требуется проектирование и реализация согласованного набора инструментов, объединяющего статический анализ и модификацию кода, а также информационное ассистирование в рамках одной архитектуры. Такая подсистема должна универсальным образом опираться на развитую модель кода CRC Python IDE, способную обеспечивать доступ как к синтаксической, так и семантической информации о программе и поддерживать безопасные операции над кодом.

2. Постановка цели и задач

Целью выпускной квалификационной работы является проектирование и реализация подсистемы для среды разработки CRC Python IDE, которая должна предоставлять инструменты (Features) для анализа и ассистирования модификаций Python-кода.

Для достижения цели были поставлены следующие задачи:

- Провести обзор инструментов для анализа и ассистирования модификаций кода в существующих средствах разработки для языка Python.
- Выполнить проектирование подсистемы, определив группы инструментов, необходимых пользователям, а также улучшения модели кода CRC Python IDE, необходимые для реализации этих инструментов.
- Выполнить реализацию спроектированной подсистемы и интегрировать ее в CRC Python IDE.
- Провести всестороннее тестирование разработанной подсистемы, модернизировав существующую тестовую инфраструктуру в проекте CRC Python IDE.
- Провести апробацию и доработку подсистемы на основе обратной связи от пользователей CRC Python IDE.

3. Обзор предметной области

Перейдем к обзору статических анализаторов, сред разработки (IDE) и систем на основе ИИ-агентов (ADE) применительно к задачам анализа и ассистирования модификаций Python-кода, чтобы сформулировать требования к проектируемой подсистеме CRC Python IDE.

3.1. Статические анализаторы

Современные статические анализаторы Python-кода опираются на стандарты типовых аннотаций и связанные с ними правила интерпретации программы. Спецификация PEP 484 [19] вводит типовые аннотации, которые упрощают статический анализ и навигацию по Python-коду, позволяя также реализовать интеллектуальное автодополнение и рефакторинг.

Муру [17] является классическим вариантом статического анализатора для Python, специализирующимся на проверке типов. Его основная задача заключается в проверке согласованности типовых аннотаций, обнаружении несоответствий между ожидаемыми и фактическими типами и раннем выявлении ошибок без выполнения программы. С инженерной точки зрения, это соответствует запросу разработчиков на диагностику проблем, связанных с типами, сигнатурами функций и связями между компонентами программы. Однако данный инструмент, в первую очередь, отвечает за информирование о проблемах в коде, но не замыкает полный цикл интерактивного исправления найденных проблем.

Pyright [14] позиционируется как более развитый статический анализатор, сочетающий два свойства, особенно значимых для интерактивных сценариев: высокую производительность и применимость к большим кодовым базам. Кроме того, Pyright сосуществует с другими сервисами в составе PyLance [13] — языкового сервера, обеспечивающего интерактивные функции среды разработки кода. На примере данного инструмента можно проследить, что анализ Python-кода постепенно эволюционировал от обособленной проверки к интеграции в более ши-

рокую инструментальную экосистему, где результаты анализа должны быть легко доступны и локализованы для конкретного места в коде.

Pyrefly [18] одновременно является как быстрым типовым анализатором, так и языковым сервером для Python с базовыми возможностями среды разработки, такими как навигация, семантическая подсветка и автодополнение. Тем самым в одном решении совмещаются диагностика, вывод типов и часть интерактивных сценариев редактора. Как и в случае с Pyright, данный инструмент стремится объединить статический анализ с полноценными возможностями среды разработки. Таким образом, статический анализ следует рассматривать не как изолированный компонент, а как основу для более широкого слоя инструментов.

Итак, можно сделать следующие выводы. Во-первых, статический анализ решает фундаментальную задачу проверки кода — выявляет ошибки типов, различные нарушения корректности использования конструкций языка, а также находит проблемы разрешения имен и логические несоответствия. Во-вторых, сам по себе анализ не покрывает все пользовательские сценарии работы с кодом. После обнаружения проблемы разработчику требуется понять ее контекст, предложить изменение и, в ряде случаев, выполнить дополнительные преобразования кода. Поэтому статический анализ в контексте CRC Python IDE должен рассматриваться как необходимая, но недостаточная основа для проектируемой подсистемы.

3.2. Среда разработки

В отличие от статических анализаторов, в IDE естественным образом объединяются статический анализ, исправления, рефакторинг, навигация и информационные ассистенты.

PyCharm [8] демонстрирует наиболее развитый вариант такой интеграции. Система инспекций обнаруживает широкий класс проблем в проекте, поддерживает профили и уровни строгости проверок, которые могут применяться как в редакторе, так и в более широком окружении для контроля качества проекта. Рефакторинг здесь является набором

надежных эквивалентных преобразований кода с настройкой параметров, предварительным просмотром результатов и проверкой конфликтов. Например, операция переименования распространяет изменение на все использования символа и поддерживает предварительный просмотр потенциально затрагиваемых мест. Данное решение подчеркивает, что для качественной разработки на Python важна согласованная работа множества инструментов поверх общей семантики программы.

На базе VSCode [15], открытой платформы для создания языковых инструментов, было разработано популярное расширение для поддержки Python, включающее языковой сервер Pylance [13] и зарекомендовавшее себя как основное среди пользователей платформы. Данный языковой сервер поддерживает глубокий статический анализ, средства исправления Python-кода, включая добавление импортов, исправление опечаток и организацию импортов. Помимо корректирующих модификаций, имеется также поддержка рефакторинга для таких сценариев, как выделение, подстановка кода и переименование с предпросмотром изменений до их применения. Данный инструмент интересен с точки зрения архитектуры, поскольку вся логика, специфичная для конкретного языка, здесь встраивается в общем виде в редактор через стандартизированный протокол Language Server Protocol (LSP) [12]. Он был разработан для того, чтобы один языковой сервер мог предоставлять различным редакторам общие языковые возможности, включая автодополнение, подсказки, навигацию, анализ и модификацию кода. Благодаря его использованию в VSCode была достигнута модульность и, как следствие, высокая масштабируемость архитектуры.

В контексте данной работы CRC Python IDE [5] рассматривается как корпоративная среда разработки для Python, основанная на платформе VSCode и расширенная внутренними сервисами и фреймворком для создания собственного пользовательского интерфейса. Она поставляется с предустановленным расширением и языковым сервером для Python, опирающимся на собственную модель кода. Однако для широкой инструментальной поддержки в CRC Python IDE необходимо проектирование и реализация целостной подсистемы на основе суще-

ствующих технологий.

Таким образом, запросы Python-разработчиков имеют комплексный характер: помимо анализа кода, необходима и поддержка исправлений, рефакторинга, а также контекстных подсказок. Поэтому в CRC Python IDE должна быть спроектирована единая подсистема, в которой статический анализ, модификация кода и информационное ассистирование будут опираться на общую модель кода.

3.3. Системы на основе ИИ-агентов

Появление систем на основе ИИ-агентов изменило сам характер взаимодействия разработчиков со средой программирования. Если традиционные IDE предоставляют пользователю отдельные действия и подсказки, то ADE стремятся решать все более разносторонние задачи в виде запросов на естественном языке: объяснять существующий код, предлагать изменения, вносить правки во множество файлов и выполнять автономные рабочие сценарии. Систематический обзор применения больших языковых моделей (LLM) в программной инженерии [11] показывает, что кодогенерация, понимание и объяснение кода, исправление и преобразование программ на сегодняшний день относятся к числу наиболее активно развивающихся направлений.

AI Assistant [7] представляет собой инструмент, развивающийся внутри сред разработки семейства IntelliJ. Он объединяет контекстно-зависимый чат, интеллектуальную помощь в редакторе, объяснение кода, предложения по рефакторингу и ИИ-агентов, способных работать с несколькими файлами одновременно. Встроенный ИИ-агент Junie позиционируется как средство автономного планирования и исполнения сложных многошаговых действий: он может изменять код, запускать тесты и команды терминала, а также использовать внешние инструменты с помощью Model Context Protocol (MCP) [16]. Одним из достоинств этого решения является наличие глубокой ИИ-поддержки прямо внутри редактора. Следовательно, теперь ИИ-ассистенты выступают не альтернативой классическим языковым сервисам, а еще одним из

потребителей семантического контекста программы.

Copilot [3] является инструментом со схожей функциональностью, но в нем особенно акцентируется внимание на диалоговые и агентные сценарии непосредственно в VSCode. Поддерживаются режимы Ask, Edit и Agent, в которых ассистент способен проводить декомпозицию задачи на шаги, читать и изменять файлы, а также работать с более широким контекстом и окружением проекта. Таким образом, у современных разработчиков есть запросы на использование ИИ-агентов для генерации кода и на выполнение управляемых модификаций в существующей кодовой базе без прямого взаимодействия с кодом.

Cursor [2] позиционируется как инструмент, прежде всего, ориентированный на глубокую интеграцию агентных сценариев в процесс разработки. Его функциональность включает фоновых ИИ-агентов, которые работают в изолированном окружении, способны самостоятельно редактировать код, запускать его и корректировать свои действия на основе оркестрации от других агентов и последующих инструкций пользователя. Это решение значительно расширяет возможности автономного выполнения задач, однако одновременно обостряет проблемы надежности и безопасности: делегирование модели широкого спектра задач повышает риск ошибок в разрешении ссылок, понимании сигнатур и определении границ безопасной модификации программы.

Ранние исследования показывают, в чем проявляются ограничения агентского подхода без использования специализированных средств обогащения контекста. В основном код, генерируемый большими языковыми моделями, подвержен галлюцинациям 5 различных типов [1]: мертвый или недостижимый код, синтаксические ошибки, логические ошибки, проблемы с надежностью и уязвимости безопасности. А в другом исследовании продемонстрировано, что модели, например, склонны использовать устаревшие API и затрудняются уточнять актуальное состояние библиотечных интерфейсов [10]. Эти результаты особенно важны для агентских сценариев, поскольку если им доступен только лишь текст файлов и вероятностная модель для продолжения работы, то качество модификаций оказывается достаточно ограничено. Следо-

вательно, для повышения надежности ИИ-агентам требуется структурированный доступ к диагностической и семантической информации о коде.

Схожие результаты подтверждаются и в более новых исследованиях, где статический анализ рассматривается как источник структурированных знаний для больших языковых моделей [6]. Из них следует, что подкрепление контекста для ИИ-агентов результатами статического анализа повышает релевантность и полноту автоматически формируемых предложений. Таким образом, в проектируемой подсистеме CRC Python IDE информационные ассистенты для ИИ-агентов должны основываться не на сыром коде проекта, а на результатах анализа и операциях, опирающихся на формальную модель кода. Поскольку CRC Python IDE уже имеет собственную модель кода, то результаты работы инструментов целесообразно предоставлять ИИ-агентам через специализированные машинно-читаемые интерфейсы.

3.4. Сравнение инструментов

Для перехода к архитектуре CRC Python IDE важно сопоставить не только функциональные возможности трех рассмотренных классов решений, но и выявить запросы Python-разработчиков, которые они удовлетворяют. Такое сравнение позволит определить требования к проектируемой подсистеме.

Таблица 1 – Сопоставление инструментов по классам средств разработки

Класс решений	Статический анализ	Модификация кода	Ассистирование
Статические анализаторы	Сильная сторона — раннее выявление ошибок и несоответствий в типах, ссылках и использовании конструкций языка	Как правило, ограничиваются сообщением о проблеме, а изменения в коде не являются их основным предназначением	Ассистирование присутствует косвенно: результаты анализа могут использоваться другими инструментами

Продолжение таблицы 1

Класс решений	Статический анализ	Модификация кода	Ассистирование
Среды разработки (IDE)	Статический анализ встроен в редактор и связан с навигацией, областями видимости и контекстом проекта	Поддерживаются корректирующие модификации и рефакторинг с предпросмотром изменений	Предоставляются контекстные подсказки, работающие поверх различных инструментов и сервисов
Системы на основе ИИ-агентов (ADE)	Статический анализ используется в качестве структурированной контекстной информации, качество которой зависит от интеграции с другими инструментами	Возможны многошаговые и многофайловые модификации, однако надежность таких сценариев ограничена без дополнительной верификации семантики программы	Сильная сторона — диалоговые и агентные сценарии, но есть риск потери актуальности контекста без структурированного доступа к модели кода

Данная таблица показывает, что статические анализаторы, IDE и ADE покрывают различные стадии цикла работы с кодом, несмотря на имеющиеся пересечения ответственности: первые концентрируются на обнаружении проблем, вторые связывают проблемы с интерактивными модификациями и контекстной поддержкой, а третьи сводят работу с кодом до диалоговых и агентных сценариев. Следовательно, при проектировании подсистемы CRC Python IDE необходимо учитывать полный цикл работы Python-разработчиков с кодом.

Таблица 2 – Выявленные особенности решений, запросы Python-разработчиков и следствия для дальнейшего проектирования подсистемы

Выявленная особенность решений	Соответствующий запрос Python-разработчиков	Следствие для дальнейшего проектирования подсистемы
Статический анализ надежно решает задачи проверки типов, ссылок, сигнатур и корректности использования конструкций Python	Находить проблемы в коде до выполнения программы и получать детальное объяснение их причины	Требуется поддержка статического анализа кода, способного выявлять проблемы до выполнения программы и объяснять их причины
Среды разработки связывают проблемы в коде с корректирующими модификациями в точке возникновения проблемы	После обнаружения проблемы получать применимое исправление без ручного изменения кода	Требуется поддержка контекстно-зависимых исправлений, применяемых в точке возникновения проблемы
Рефакторинг в средах разработки является самостоятельным классом эквивалентных преобразований кода с предпросмотром и контролем области изменений	Выполнять более общие преобразования кода, улучшающие структуру программы и поддерживаемость проекта	Требуется поддержка рефакторинга кода, управляемого на уровне семантики программы
Подсказки превращают результаты анализа в контекстно-зависимую информацию, доступную во время редактирования	Быстро получать сведения о декларациях, типах, сигнатурах и использовании объектов непосредственно в редакторе	Требуется поддержка информационного ассистирования для пользователей, предоставляющего контекстные сведения о декларациях, типах, сигнатурах и использовании объектов непосредственно в редакторе

Продолжение таблицы 2

Выявленная особенность решений	Соответствующий запрос Python-разработчиков	Следствие для дальнейшего проектирования подсистемы
ИИ-агенты способны выполнять комплексные сценарии модификации кода, но без структурированного семантического контекста их надежность ограничена	Использовать ИИ-агентов при работе с кодовой базой с минимальными потерями точности контекста и управляемостью изменений	Требуется поддержка информационного ассистирования для ИИ-агентов в виде структурированного семантического доступа к сведениям о коде и безопасным операциям по его модификации

Данная таблица переводит выявленные особенности классов решений на язык потребностей Python-разработчиков, а следствия из них показывают, какие пользовательские и агентные сценарии необходимы для проектируемой подсистемы CRC Python IDE.

Таким образом, в ходе обзора были рассмотрены ключевые особенности статических анализаторов, IDE и ADE, а также выявлены соответствующие им запросы Python-разработчиков. На их основе были определены требования к проектируемой подсистеме CRC Python IDE: выявление проблем в коде, получение детализированного объяснения их причин, применение корректирующих модификаций, выполнение рефакторинга кода, получение контекстной информации о декларациях, типах и сигнатурах в редакторе, а также предоставление ассистентов для ИИ-агентов, сохраняющих семантическую точность и управляемость изменений кода.

4. Архитектура

Результаты обзора предметной области показали, что статические анализаторы обеспечивают раннее выявление проблем в коде, но не замыкают полный цикл их устранения непосредственно в редакторе, среды разработки связывают анализ кода с модификацией программы и контекстной поддержкой пользователя, а системы на основе ИИ-агентов сводят цикл разработки до диалоговых и автономных сценариев, однако предъявляют повышенные требования к точности семантической информации и управляемости изменений. Поэтому при проектировании подсистемы CRC Python IDE необходимо перейти от перечня отдельных возможностей к согласованной архитектуре, системно покрывающей выявленные запросы пользователей.

4.1. Инструменты подсистемы

На основании обзора предметной области можно выделить три основные группы возможностей: инструменты статического анализа кода, инструменты модификации кода и инструменты информационного ассистирования. Такая декомпозиция соответствует не только формальной классификации по общим признакам самих инструментов, но и структуре пользовательских сценариев работы с кодом.

Выбор инструментов для подробного изложения при проектировании подсистемы CRC Python IDE подчинен двум критериям. Во-первых, каждый из инструментов должен покрывать не один частный случай, а целый класс пользовательских или агентных сценариев. Во-вторых, он должен быть архитектурно показательным, то есть затрагивать основные свойства подсистемы: работу с синтаксической и семантической информацией, связь результатов статического анализа с модификацией программы, корректность преобразований и повторное использование модели кода.

По этой причине для дальнейшего описания выбираются наиболее репрезентативные и архитектурно значимые инструменты, а полный перечень спроектированных инструментов вынесен в приложение.

Таблица 3 – Архитектурная структура подсистемы и репрезентативные инструменты

Группа	Подгруппа	Архитектурная роль	Репрезентативные инструменты
Инструменты статического анализа кода	Статический анализ кода	Формирование диагностического слоя подсистемы: от базовой проверки корректности текста программы до анализа ссылок, типов и потока выполнения	Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code
Инструменты модификации кода	Исправление кода	Применение контекстно-зависимых корректирующих модификаций, устраняющих выявленную проблему и восстанавливающих недостающий элемент программы или окружения	Install Package, Import Reference, Create Reference, Add Qualifier
Инструменты модификации кода	Рефакторинг кода	Выполнение эквивалентных преобразований, изменяющих структуру программы при сохранении ее семантики	Extract, Inline, Rename, Optimize Imports
Инструменты информационного ассистирования	Информационные ассистенты для пользователей	Предоставление пользователю контекстной информации о декларациях, типах, сигнатурах и использовании элементов программы непосредственно в редакторе	Hover, Signature Help, Inlay Hints
Инструменты информационного ассистирования	Информационные ассистенты для ИИ-агентов	Предоставление агенту контекста в виде диагностических и семантических данных, а также безопасных операций над кодом	Problems MCP, Hover MCP, Signature Help MCP, Rename MCP

В группе статического анализа кода выбранные инструменты образуют последовательность от минимально необходимой синтаксической проверки программы к более сложным семантическим верификациям.

Инструмент Syntax Correctness выбран первым, поскольку показывает самый базовый уровень состоятельности исходного текста программы и тем самым задает нижнюю границу применимости всех последующих инструментов: до тех пор, пока программа синтаксически некорректна, более глубокий анализ может быть неточен или вовсе невозможен. Следующий инструмент — Reference Resolve, поскольку именно анализ разрешимости импортов, простых и квалифицированных ссылок связывает диагностику проблем с навигацией и многими корректирующими модификациями. Затем следует инструмент Expected Type, демонстрирующий роль вывода типов и сопоставления ожидаемого и фактического типов выражения. Завершает этот ряд инструмент Unreachable Code как показательный пример анализа, работающего уже не только с локальной конструкцией в коде, но и со свойствами потока выполнения программы. В совокупности эти четыре инструмента образуют стандартный компактный набор для статического анализа кода: синтаксическая корректность, разрешение ссылок, типовая интерпретация и анализ исполнимости.

Группа инструментов модификации кода принципиально неоднородна, поэтому внутри нее необходимо отделить исправление кода от рефакторинга. Исправления ориентированы на восстановление корректного состояния программы в точке возникновения проблемы. Для их представления выбраны инструменты Install Package, Import Reference, Create Reference и Add Qualifier. Инструмент Install Package показывает, что корректирующая модификация может выходить за пределы редактирования файлов проекта и затрагивать окружение разработки. Инструмент Import Reference представляет прямую связь между определением неразрешенной ссылки и импортом ссылки для ее исправления. Инструмент Create Reference демонстрирует сценарий, в котором подсистема не только восстанавливает доступность существующего элемента программы, но и синтезирует недостающую сущность с учетом контекста ее использования. Наконец, инструмент Add Qualifier представляет исправление, основанное не на создании нового элемента, а на семантическом уточнении способа обращения к уже существующему.

Таким образом, выбранные инструменты совместно покрывают четыре базовых сценария корректирующих действий: работу с окружением, восстановление импортной связности, генерацию недостающих определений и уточнение контекста обращения.

Группа инструментов рефакторинга, в отличие от исправлений кода, не всегда основывается на результатах анализа кода и потому требует отдельного набора репрезентативных инструментов. Инструменты Extract, Inline, Rename и Optimize Imports выбраны как классические и одновременно архитектурно значимые преобразования. Инструмент Extract представляет выделение новой сущности из существующего кода и тем самым показывает один из базовых сценариев эквивалентного преобразования. Инструмент Inline демонстрирует обратное по направлению преобразование, связанное с упрощением структуры программы. Инструмент Rename является важным примером семантически корректного изменения, поскольку требует согласованной работы между определением идентификатора и нахождением всех его использований. Инструмент Optimize Imports показывает связь статического анализа, модификации и последующей нормализации структуры кода на уровне импортов. Вместе эти инструменты отражают основные направления эквивалентных преобразований: выделение, сворачивание, переименование и нормализацию структуры кода.

Группа инструментов информационного ассистирования тоже естественным образом представляется через две подгруппы. Первая подгруппа ориентирована на пользователя и обеспечивает контекстную интерпретацию программы непосредственно в процессе редактирования. Здесь в качестве репрезентативных инструментов выбраны Hover, Signature Help и Inlay Hints. Инструмент Hover предоставляет точечный доступ к сведениям о декларациях, типах и документации. Инструмент Signature Help отображает подсказку для сигнатур в контексте конкретного вызова функции. Inlay Hints дополняет их как средство встроенного представления семантической информации непосредственно в коде.

Вторая подгруппа информационных ассистентов ориентирована на ИИ-агентов. Набор инструментов Problems MCP, Hover MCP, Signature

Help MCP и Rename MCP показывает, как результаты анализа, сведения о типах и сигнатурах, а также безопасные операции над кодом переводятся из пользовательского представления в машинно-читаемое. В результате информационные ассистенты поддерживают два разных способа потребления одной и той же семантической информации — пользовательский и агентный.

Теперь, когда определены все подгруппы и репрезентативные инструменты, необходимо рассмотреть, на какой общей инфраструктуре они будут реализованы. Основу этой инфраструктуры образует модель кода CRC Python IDE, от выразительности и стабильности которой зависят корректность статического анализа кода, возможность программных модификаций и качество контекстной семантической информации.

4.2. Улучшения модели кода

Для реализации выделенных в предыдущем разделе групп инструментов требуется единая и достаточно выразительная модель кода, обеспечивающая доступ к синтаксической и семантической информации о программе. Такая модель кода должна не только отражать структуру исходного текста программы, но и поддерживать разрешение ссылок, получение сведений о типах, а также безопасные программные модификации кода.

Существующая модель кода CRC Python IDE обеспечивала описанную функциональность, достаточную для реализации отдельных сценариев, однако ее возможностей было недостаточно для построения целостной подсистемы для выделенных групп инструментов. В частности, требовались более унифицированный доступ к элементам программы, более гибкое разрешение ссылок в зависимости от контекста и более удобный интерфейс для получения структурированной информации о типах.

Поэтому в рамках работы были выполнены целенаправленные инфраструктурные доработки существующей модели кода. Они были направлены на расширение и стабилизацию синтаксического и семантиче-

ского представления программы (PSI), уточнение и конфигурирование возможностей разрешения ссылок (Resolve), а также доработку интерфейса вывода типов (Type Inference). Изменения в этих компонентах направлены на подготовку инфраструктурной основы для инструментов анализа, исправления и рефакторинга кода, а также информационных ассистентов.

4.2.1. Расширение и стабилизация синтаксического и семантического представления программы

Подсистема инструментов CRC Python IDE опирается на синтаксическое и семантическое представление программы, реализованное в виде дерева PSI [24]. Это представление отражает структуру исходного кода и предоставляет доступ к его элементам. Исходная реализация PSI обеспечивала основные возможности синтаксического анализа, однако для поддержки расширенного набора инструментов требовался более унифицированный и устойчивый доступ к конструкциям языка Python, а также более надежные средства программной модификации дерева.

Одним из ключевых направлений доработок стало развитие интерфейсной модели узлов PSI-дерева. Были введены специализированные интерфейсы, обеспечивающие унифицированный доступ к различным видам синтаксических конструкций, таким как идентификаторы, типовые аннотации, декораторы, комментарии, документационные строки и списки параметров. Это позволило исключить дублирование логики работы с деревом в инструментах и перейти к более единообразной работе с элементами программы.

Для учета особенностей языка Python были реализованы вспомогательные классы, инкапсулирующие типовые операции над синтаксическими элементами. К таким операциям относятся проверка корректности идентификаторов с учетом соглашений языка, анализ и классификация параметров функций, а также работа со строковыми литералами, используемыми в качестве носителей семантически значимой информации. В результате инструменты перестали зависеть от множества локальных эвристик и получили более устойчивую основу на уровне

общей инфраструктуры PSI.

Еще одним направлением доработок стало повышение устойчивости операций, связанных с программной модификацией PSI-дерева. Были добавлены механизмы, стабилизирующие изменение структуры дерева с учетом синтаксических особенностей Python: автоматическую коррекцию форматирования и отступов при вставке и удалении узлов, корректное обновление связей между элементами, а также безопасное построение новых синтаксических конструкций на основе вспомогательных фабрик. Благодаря этому PSI стал надежен не только для чтения, но и для контролируемого изменения кода.

Дополнительно были реализованы средства для работы с импортами, обеспечивающие корректную модификацию импортных блоков без нарушения их последовательности и форматирования. В совокупности выполненные доработки PSI создают прочную основу для работы всех типов инструментов подсистемы: статический анализ получает более удобный доступ к структуре программы, исправления и рефакторинги — надежные механизмы программной модификации, а информационные ассистенты — стабильную основу для контекстной интерпретации кода.

При этом выполненные изменения не затрагивают базовую архитектуру PSI-дерева и механизмы синтаксического анализа. Они сосредоточены на уровне интерфейсов, вспомогательных классов и механизмов модификации, расширяя возможности существующей модели кода.

4.2.2. Уточнение и конфигурирование возможностей разрешения ссылок

Подсистема инструментов CRC Python IDE требует гибкого и контекстно-зависимого доступа к семантической информации о программе. Ключевым элементом такого доступа является механизм разрешения ссылок, обеспечивающий сопоставление идентификаторов с соответствующими декларациями. Реализация Resolve [27] покрывала основные сценарии анализа потока выполнения программы, однако для поддержки расширенного набора инструментов требовалась более тон-

кая настройка поведения в зависимости от контекста применения.

Одним из основных направлений доработок стало введение механизма стратегий разрешения идентификаторов. Он позволяет варьировать поведение Resolve в зависимости от контекста применения, включая предварительную обработку результатов, основную логику разрешения и последующую фильтрацию. Реализованные стратегии дали возможность адаптировать один и тот же механизм к разнообразным задачам без дублирования семантической логики.

Для более точного контроля процесса разрешения был введен механизм условий остановки, определяющий, на каких типах деклараций следует завершать поиск. Это позволяет настраивать глубину и область разрешения в зависимости от решаемой задачи. Комбинирование условий остановки делает возможным ограничивать Resolve нужным уровнем детализации и тем самым избегать как преждевременного завершения, так и избыточного обхода ссылочного графа программы.

Отдельным направлением стало обеспечение разрешения ссылок, представленных в строковых литералах. Подобные случаи возникают, в частности, в типовых аннотациях, атрибутах классов и специальных конструкциях языка. Для их поддержки была реализована система провайдеров, выполняющих извлечение идентификаторов из строкового контекста и последующее разрешение по общим правилам. Это позволило распространить возможности Resolve на сценарии, которые ранее требовали локальных решений или вовсе не поддерживались.

В рамках доработок были добавлены и специализированные вспомогательные механизмы, обеспечивающие корректную обработку отдельных языковых конструкций и сценариев использования, в том числе получение всех объявлений перегруженных функций, различные режимы разрешения перегрузок и корректное разрешение квалифицированных ссылок в сочетании с типовой информацией. В совокупности эти изменения сделали Resolve не только более гибким, но и пригодным для повторного использования в разных частях подсистемы.

Выполненные доработки не изменяют фундаментальные алгоритмы разрешения ссылок, а представляют собой надстройки над существую-

щей системой. Они расширяют возможности ее конфигурирования, сохраняя обратную совместимость и позволяя использовать разрешение ссылок как общий механизм для различных групп инструментов.

4.2.3. Доработка интерфейса вывода типов

Подсистема инструментов CRC Python IDE требует доступа к структурированной информации о типах программных элементов. Такая информация используется при анализе корректности кода, генерации модификаций и формировании пользовательских подсказок. Существующая система Type Inference [25][26] обеспечивала возможности для точного вывода типов, однако ее интерфейс был недостаточно удобен и однороден для удовлетворения всех требований подсистемы.

В рамках работы были выполнены доработки, направленные на расширение и унификацию интерфейса вывода типов. Это позволило обеспечить доступ к семантической информации без необходимости взаимодействия с внутренними деталями реализации типовой системы. Type Inference в CRC Python IDE основан на сочетании абстрактной интерпретации выражений и отдельного слоя провайдеров, вычисляющих типовую информацию для специфичных элементов программы.

Одним из основных направлений стало расширение базового интерфейса представления типов. В него были добавлены средства, позволяющие получать сведения об атрибутах типов, извлекать типы элементов итерируемых и других специальных объектов, а также работать с параметризованными типами. Дополнительно введены методы для уточнения типовой информации, благодаря которым инструменты получают не исходный результат работы алгоритма, а более детальное представление типов, когда это необходимо.

Для поддержки сценариев, связанных с анализом вызовов функций, был доработан механизм обработки перегрузок. Обновленный интерфейс обеспечивает сопоставление аргументов с параметрами функций с учетом различных вариантов сигнатур, включая частично примененные функции, а также позволяет работать с неполными или неоднозначными случаями. Это существенное улучшение для инструментов

подсистемы, связанных с анализом вызовов функций.

Отдельное внимание было уделено унификации представления типов для множества сценариев использования. Расширенный механизм форматирования типов позволяет получать их представление как в виде, ориентированном на пользователя, так и в машинно-читаемом виде, пригодном для генерации кода. Поддерживается настройка отображения вложенности, параметров, объединений и иных типовых конструкций, что делает один и тот же источник типовой информации пригодным как для анализа и подсказок, так и для действий над кодом.

Выполненные изменения не затрагивают фундаментальные алгоритмы вывода типов, они сосредоточены на уровне интерфейсов и точек расширения, обеспечивая более удобный и универсальный доступ к информации о типах. Таким образом, Type Inference становится общей точкой опоры для инструментов, работающих с семантикой выражений, сигнатурами вызовов и типовой интерпретацией программных элементов.

Теперь, когда рассмотрены все группы инструментов и улучшения модели кода, можно перейти к их проектированию на уровне архитектуры CRC Python IDE.

4.3. Структура подсистемы

В предыдущих подразделах были рассмотрены группы инструментов, образующие содержательный состав подсистемы CRC Python IDE, а также инфраструктурные доработки модели кода, делающие возможной реализацию инструментов на единой основе. Теперь необходимо объединить эти результаты и показать, как инструменты статического анализа кода, модификации и информационного ассистирования объединяются в единую подсистему.

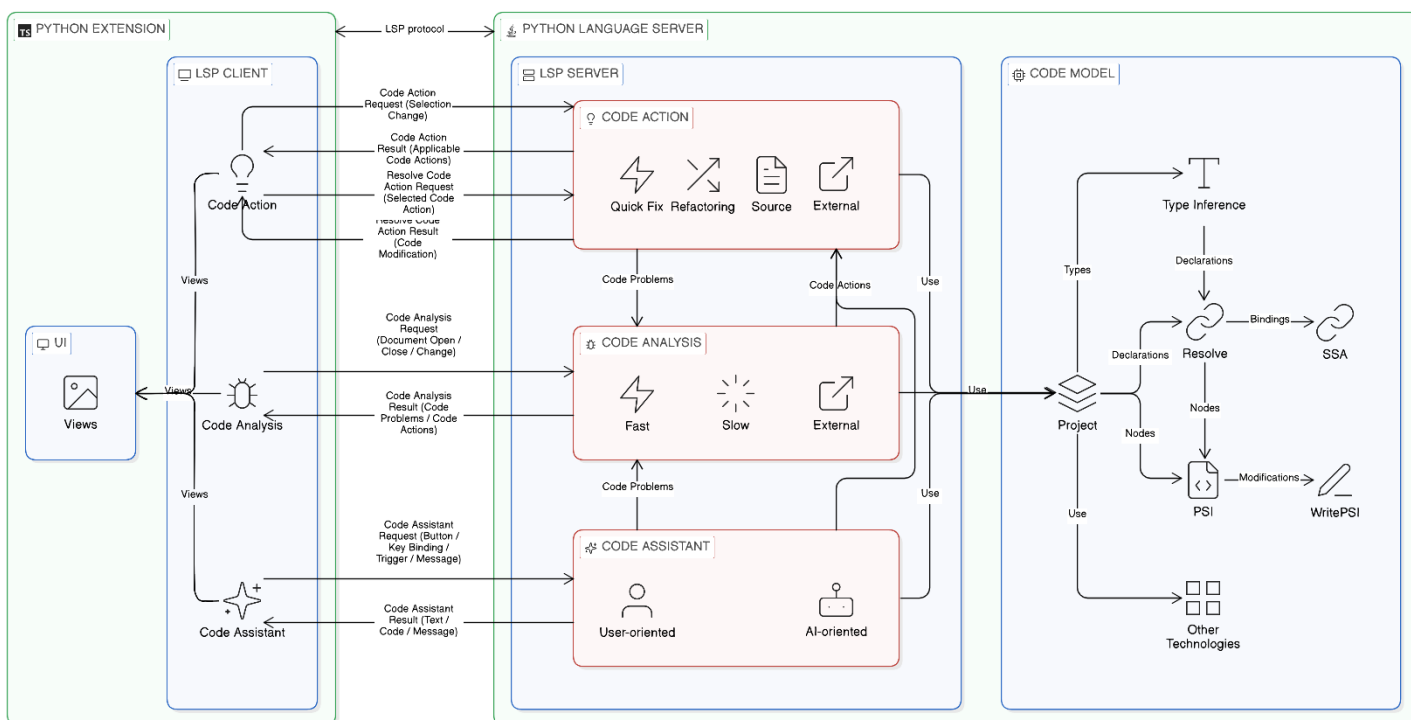


Рис. 1: Структура подсистемы.

Диаграмма показывает архитектуру CRC Python IDE, которая строится вокруг трех взаимосвязанных компонентов: Python Extension, Python Language Server и Code Model. Такое разбиение отвечает за распределение ответственности между расширением среды разработки, языковым сервером и моделью кода. Пользовательские и агентные сценарии инициируются в Python Extension, обработка клиентских запросов и координация инструментов сосредоточены в Python Language Server, а общая семантическая основа находится в Code Model. Благодаря этому выделенные группы инструментов оказываются связаны через единую архитектуру.

В компоненте Python Extension есть две ключевые подсистемы — UI и LSP Client. UI отображает пользовательский интерфейс, в котором представлены соответствующие Views для взаимодействия с результатами анализа, действиями над кодом и ассистирующими сценариями. LSP Client выступает посредником между платформой VSCode и языковым сервером. Именно через него инициируются запросы Code Analysis, Code Action и Code Assistant, а затем возвращаются результаты в форме диагностик, модификаций кода и ассистирующих ответов. Таким

образом, Python Extension обеспечивает возможность взаимодействия с инструментами подсистемы, но не отвечает за семантическую обработку программы.

Основная логика работы инструментов расположена в компоненте Python Language Server. LSP Server обеспечивает протокольное взаимодействие с клиентской частью по LSP, образуя платформу для всех частей подсистемы — Code Analysis, Code Action и Code Assistant. Code Analysis образует диагностический слой подсистемы и поддерживает асинхронные коллекции Fast, Slow и External для эффективного предоставления результатов анализа. Code Action представляет слой действий над кодом и включает Quick Fix, Refactoring, а также Source и External как дополнительные категории действий над кодом. Слой Code Assistant объединяет оба направления информационного ассистирования — User-oriented и AI-oriented.

Связи между Code Analysis, Code Action и Code Assistant отражают рабочий цикл подсистемы. Code Analysis формирует Code Problems, которые используются как основа для корректирующих действий в Code Action и как часть контекста в Code Assistant. Code Action, в свою очередь, возвращает Code Modifications и тем самым переводит результаты диагностики в управляемые изменения кода программы. Code Assistant опирается на тот же семантический контекст, но предоставляет его либо пользователю, либо ИИ-агенту в соответствующем формате. Кроме того, все части подсистемы связаны взаимными обращениями и общим использованием Code Model.

Инфраструктурная основа подсистемы сосредоточена в компоненте Code Model. Центральным элементом здесь выступает Project, который задает общий проектный контекст для всех остальных механизмов. С ним связаны PSI, Resolve, Type Inference, SSA, WritePSI и Other Technologies. PSI отвечает за синтаксическое и семантическое представление программы на уровне узлов дерева и структур для работы с исходным кодом. Resolve обеспечивает связывание деклараций и использований, а SSA дополняет его более точной интерпретацией определений и обращений. Type Inference предоставляет сведения о типах и опира-

ется на Resolve. WritePSI показывает, что модель кода поддерживает не только структурированное чтение программы, но и ее программную модификацию. Other Technologies позволяет подключение дополнительных языковых сервисов, использующих тот же проектный контекст, например, систем управления зависимостями. Таким образом, повторно используемая инфраструктура Code Model обеспечивает целостность подсистемы CRC Python IDE, подтверждая, что доработки PSI, Resolve и Type Inference являются важным условием для реализации всех выделенных групп инструментов.

Рассмотрев многоуровневую архитектуру CRC Python IDE и структуру спроектированной подсистемы, перейдем к реализации описанных инструментов в рамках каждой подгруппы.

5. Реализация

В архитектуре были зафиксированы: общая структура подсистемы CRC Python IDE и полный состав групп инструментов, а также инфраструктурные доработки модели кода, которые сделали возможной их согласованную работу. Теперь рассмотрим, каким образом спроектированное решение было воплощено на уровне конкретной функциональности среды разработки. В практической реализации архитектурное разделение на инструменты статического анализа кода, модификации кода и информационного ассистирования раскрывается более гранулярно через пять подгрупп: инструменты для статического анализа кода, инструменты для исправления кода, инструменты для рефакторинга кода, инструменты информационного ассистирования для пользователей и для ИИ-агентов.

Полный перечень реализованных инструментов вынесен в приложение, поэтому в настоящем разделе внимание будет сфокусировано на наиболее показательных и архитектурно значимых примерах. В каждой подгруппе рассматриваются инструменты, которые одновременно являются базовыми для множества сценариев, затрагивают разные уровни работы с Python-кодом и позволяют увидеть, как единая модель кода, механизмы разрешения ссылок, вывод типов и контролируемые операции изменения кода используются в рамках согласованной подсистемы.

5.1. Инструменты для статического анализа кода

Подгруппа инструментов для статического анализа кода образует диагностический слой всей подсистемы. Она организована как семейство статических проверок, в которой отдельные инспекции не существуют изолированно, а подключаются к общей инфраструктуре регистрации, обхода PSI-дерева, накопления результатов и публикации диагностик в LSP-формате. За счет этого в одной архитектуре удастся совместить как быстрые проверки, работающие незамедлительно по синтаксическому представлению программы, так и более глубокие семантические ана-

лизы, опирающиеся на разрешение ссылок, сведения о типах и свойства потока выполнения. Для дальнейшего рассмотрения выбраны Syntax Correctness, Reference Resolve, Expected Type и Unreachable Code, размещенные в порядке нарастания аналитической сложности.

Инструмент Syntax Correctness задает нижнюю границу применимости всей подгруппы. Его задача состоит не в дублировании логики парсера, а в переводе синтаксических ошибок, обнаруженных на этапе разбора, в общий диагностический формат подсистемы. Такой анализ необходим по двум причинам. Во-первых, синтаксическая корректность становится полноправной частью аналитического слоя, а не предварительной проверкой на уровне модели кода. Во-вторых, анализ синтаксиса сохраняет единый жизненный цикл с остальными инструментами: он привязан к конкретным узлам ошибок, отображается через общие механизмы и может участвовать в дальнейшем связывании с корректирующими действиями. Репрезентативность этого инструмента заключается в том, что он показывает, как статический анализ, выполняемый еще при построении модели кода, встраивается в общую архитектуру подгруппы без специальных обходных путей.

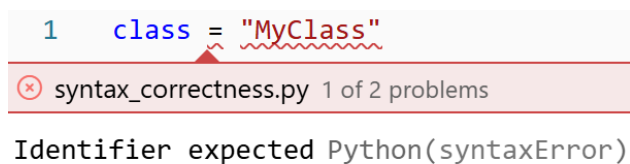


Рис. 2: Пример работы инструмента Syntax Correctness в случае ошибки присваивания значения в ключевое слово.

Инструмент Reference Resolve представляет центральный семантический анализатор подгруппы, поскольку с помощью него решаются задачи интерпретации импортов, простых имен и квалифицированных обращений. Его реализация организована как семейство взаимосвязанных проверок, объединенных общим контрактом анализа ссылок, но различающихся по типу целевой конструкции. Это позволяет обрабатывать различные случаи разрешения имен в единой архитектуре: для импортов требуется учитывать доступность модулей и элементов в проектном и внешнем окружении, для простых ссылок — области видимо-

сти и подходящие определения, для квалифицированных — семантику объекта, стоящего слева от обращения. Важно, что Reference Resolve опирается не только на синтаксическую информацию, но и на SSA-ориентированное разрешение ссылок, индексы проекта и, в квалифицированных случаях, на вывод типов. За счет этого данный инструмент связывает базовую структуру программы с более глубокими слоями ее семантической интерпретации и одновременно подготавливает основу для последующих корректирующих модификаций, таких как добавление импорта, создание недостающей сущности и уточнение способа обращения.

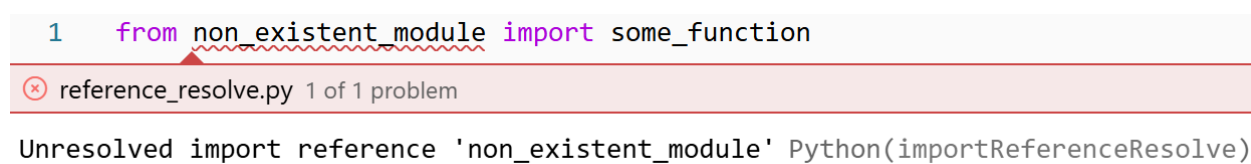


Рис. 3: Пример работы инструмента Reference Resolve в случае ошибки использования несуществующей ссылки в импорте.

Инструмент Expected Type переводит анализ на уровень сопоставления фактической и ожидаемой семантики выражения. В отличие от проверки разрешимости имени, здесь уже недостаточно установить, к какому элементу относится конструкция — требуется определить, какой тип ожидается в данной точке программы и насколько он согласуется с типом фактического выражения. Поэтому реализация этого инструмента строится как система специализированных провайдеров для нахождения ожидаемого типа, охватывающих разные контексты использования: аргументы вызовов, параметры со значениями по умолчанию, аннотированные присваивания, литералы коллекций и генераторные конструкции. Такая организация показывает, что подсистема статического анализа работает не только с уже вычисленным типом выражения, но и с его ролью в программном окружении. Этот инструмент также открывает простор для корректирующих модификаций, связанных с преобразованием типовых аннотаций.

```
1  from typing import List, Dict, Optional
2
3
4  def process_data(data: List[Dict[str, int]]) -> Optional[int]:
5      if not data:
6          return None
7
8      total = 0
9      for item in data:
10         for _, value in item.items():
11             total += value
12
13     return total
14
15
16  data = [{"a": "1", "b": "2"}]
17  process_data(data)
```

⚠ type_checking.py 1 of 1 problem

Expected type 'list[dict[str, int]]', but got 'list[dict[str, str]]' Python(expectedType)

Рис. 4: Пример работы инструмента Expected Type в случае предупреждения о несовпадении типов аргумента и параметра функции.

Инструмент Unreachable Code завершает выбранный набор как пример анализа, ориентированного уже не на отдельные имена и типы, а на управляющую структуру программы. Его задача состоит в обнаружении фрагментов, которые гарантированно не могут быть достигнуты в текущем потоке выполнения. Для этого используются правила определения конструкций, завершающих исполнение текущего контекста, и последующая проверка списков операторов с учетом условных ветвлений, циклов, обработки исключений, блоков финализации и контекстных менеджеров. Особенность этого инструмента состоит в том, что он работает с общими свойствами потока управления программы. За счет этого Unreachable Code является одним из нетривиальных примеров и показывает, что подгруппа статического анализа CRC Python IDE охватывает не только синтаксис, ссылки и типы, но и интерпретацию потока выполнения программы.


```
1  for i in range(10):
2      if i == 5:
3          continue
4          This code is unreachable Python(unreachableCode)
5          No quick fixes available
6          print("Skip")
7
8
9  print(i)
```

Рис. 5: Пример работы инструмента Unreachable Code в случае информирования о недостижимом коде после перехода к следующей итерации цикла.

Устройство данной подгруппы принципиально важно и для дальнейшей реализации: именно статический анализ формирует набор семантически значимых фактов, на который затем опираются инструменты исправления кода, связывающие диагностику с контролируемыми модификациями программы.

5.2. Инструменты для исправления кода

Инструменты для исправления кода образуют первую подгруппу внутри инструментов модификации кода. Их задача состоит в том, чтобы преобразовать результат диагностики в корректирующее действие, устраняющее причину проблемы либо в самом тексте программы, либо в проектном окружении. По этой причине исправления занимают промежуточное положение между анализом и преобразованием кода: они инициируются конкретной диагностикой, но реализуются через общие механизмы изменения программы, обновления конфигураций и внешних зависимостей. В CRC Python IDE эта подгруппа организована как единый архитектурный слой, где исправления централизованно регистрируются, создаются через общий фабричный механизм, проверяют применимость в контексте конкретной проблемы и затем используют унифицированные способы внесения изменений. Для подробного рассмотрения выбраны Install Package, Import Reference, Create Reference и Add Qualifier — вместе они представляют четыре базовых класса корректирующих модификаций: работу с окружением, восстановление им-

портной связности, создание недостающей сущности и уточнение доступа к существующему элементу программы.

Инструмент Install Package показывает наиболее широкий по охвату сценарий исправления. В ряде случаев неразрешенный импорт нельзя устранить одним только редактированием файла, поскольку источник проблемы находится на уровне отсутствующей зависимости. Поэтому инструмент работает не с локальным фрагментом кода, а с окружением всего проекта: по диагностике неразрешенного импорта он определяет предполагаемое имя пакета, сопоставляет его с известными источниками пакетов и затем делегирует установку подсистеме управления зависимостями. Архитектурная ценность этого инструмента состоит в том, что он расширяет понятие корректирующей модификации: исправление может означать не только переписывание кода, но и восстановление полноты окружения, в которой этот код существует.



Рис. 6: Пример работы инструмента Install Package в случае установки пакета по ссылке в импорте.

Инструмент Import Reference представляет типичный случай локального исправления, непосредственно связанного с диагностикой

неразрешенной ссылки. Однако даже такой, на первый взгляд, простой инструмент требует значительного объема семантической работы. Сначала необходимо установить роль идентификатора в текущем контексте, затем найти возможные источники импорта в пределах проекта и зависимостей, а после этого выбрать способ вставки, согласованный с областью видимости и местом использования элемента. Если найдено несколько возможных источников, система должна не просто показать их пользователю, а предварительно ранжировать варианты на основе статистики так, чтобы на первом месте оказывались наиболее вероятные соответствия. Именно поэтому Import Reference выступает репрезентативным примером связи между диагностикой, разрешением ссылок, пользовательским интерфейсом и контролируемым редактированием программы.

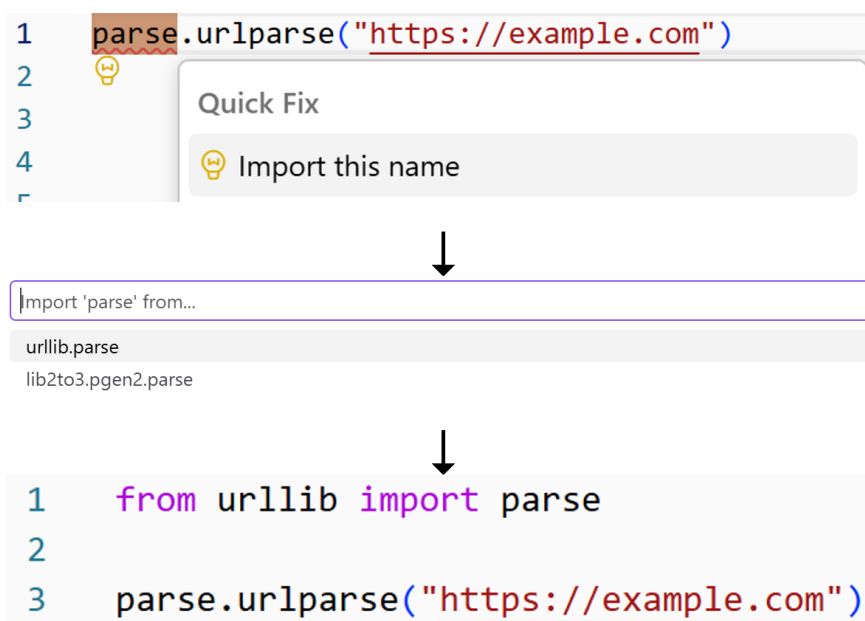


Рис. 7: Пример работы инструмента Import Reference в случае импорта ссылки по идентификатору.

Инструмент Create Reference демонстрирует следующий уровень исправления, поскольку применяется, когда проблему невозможно устранить подключением уже существующей сущности. В таких случаях система должна построить недостающий элемент программы на основе контекста использования. По этой причине инструмент реализован как семейство исправлений для создания классов, функций, методов, полей,

переменных и параметров. Архитектурно важен не сам факт генерации новой сущности, но и способ ее построения: по месту использования система выводит, какой вид определения требуется, какие атрибуты должны быть заданы сразу, в каком модуле новое определение должно быть создано и какие элементы интерфейса или сигнатуры можно восстановить из контекста. Этот инструмент является показательным для всей подгруппы, поскольку он демонстрирует генерацию нового кода, сохраняя при этом связь с конкретной диагностикой.

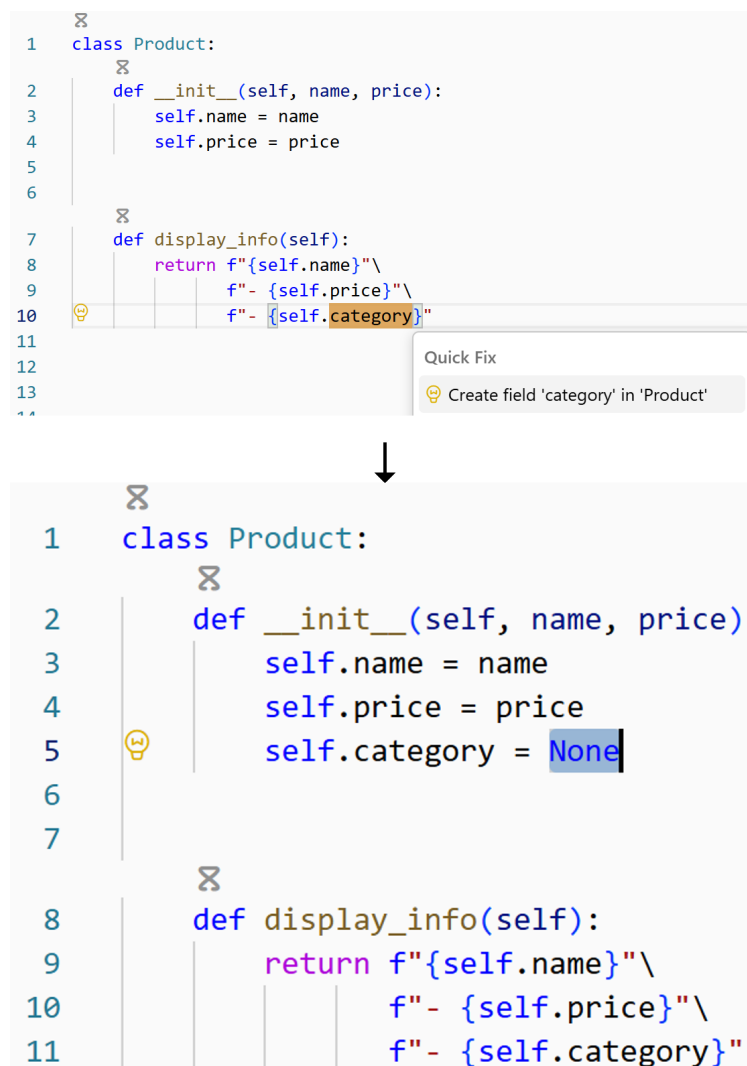


Рис. 8: Пример работы инструмента Create Reference в случае создания поля по идентификатору.

Инструмент Add Qualifier представляет наиболее точный тип исправления. Здесь проблема состоит не в том, что нужная сущность отсутствует, и не в том, что не подключена необходимая зависимость, а

в том, что уже существующий элемент программы используется через неправильный способ доступа. Для корректного исправления необходимо определить контекст использования идентификатора и выбрать квалификатор, который делает обращение осмысленным: “self”, “cls”, имя класса или “super”. Это требует анализа охватывающей функции или метода, положения ссылки в классе, доступности атрибутов и, в ряде случаев, учета иерархии наследования. Тем самым Add Qualifier показывает, что даже точечная корректирующая модификация может опираться на весьма глубокую семантическую интерпретацию программы.



Рис. 9: Пример работы инструмента Add Qualifier в случае добавления квалификатора обращения к атрибуту суперкласса.

Рассмотренные инструменты показывают, что исправление кода CRC Python IDE реализовано как самостоятельный архитектурный слой, опирающийся на результаты анализа, но не исчерпывающийся простыми текстовыми заменами. Это подводит реализацию к следующей подгруппе — инструментам для рефакторинга кода, в которых модификация уже не обусловлена конкретной диагностикой, а используется как средство более независимого эквивалентного преобразования программы.

5.3. Инструменты для рефакторинга кода

Инструменты для рефакторинга кода занимают особое место внутри группы инструментов модификации кода. В отличие от исправлений, которые реагируют на конкретную найденную проблему, рефакторинг ориентирован на прямое преобразование структуры программы при сохранении ее внешнего поведения. Это различие определяет и архитектуру подгруппы. Если для исправления достаточно связать диагностику с корректирующим действием, то рефакторинг требует более строгой проверки применимости, выявления затрагиваемых элементов программы, поиска связанных использований и формирования согласованного набора изменений, которые могут быть безопасно применены с помощью модели кода. В CRC Python IDE эта подгруппа строится вокруг общей инфраструктуры действий над кодом, где логика проверки условий преобразования отделена от логики самого переписывания программы. Для подробного рассмотрения выбраны инструменты Extract, Inline, Rename и Optimize Imports, поскольку они покрывают четыре принципиально разных, но архитектурно взаимосвязанных сценария: выделение, сворачивание, переименование и нормализацию структуры кода.

Инструмент Extract наглядно демонстрирует рефакторинг как построение нового элемента программы на основе уже существующего кода. В реализации он объединяет два тесно связанных механизма: извлечение новой функции или метода и введение новой переменной. В

обоих случаях ключевым этапом становится не само создание новой сущности, а предварительный анализ выделенного фрагмента. Система должна корректно определить границы выражения или группы операторов, установить зависимости от внешнего контекста, понять, какие значения следует передавать через параметры, и только после этого сформировать план преобразования. На практике это означает, что Extract работает как управляемая реконструкция существующего фрагмента программы.

```

1 def process_user_data_before(user_data):
2     full_name = get_full_name(user_data)
3
4     street = user_data.get('address', {}).get('street', '').strip()
5     city = user_data.get('address', {}).get('city', '').strip()
6     zip_code = user_data.get('address', {}).get('zip_code', '')
7     formatted_address = f"{street}, {city} {zip_code}"
8
9     return {
10         'name': full_name,
11         'address': formatted_address
12     }
13
14
15 def get_full_name(user_data):
16     first_name = user_data.get('first_name', '').strip().title()
17     last_name = user_data.get('last_name', '').strip().title()
18     full_name = f"{first_name} {last_name}"
19     return full_name

```

↓

```

1 def process_user_data_before(user_data):
2     full_name = get_full_name(user_data)
3     formatted_address = new_name(user_data)
4
5     return {
6         'name': full_name,
7         'address': formatted_address
8     }
9
10
11 def get_full_name(user_data):
12     first_name = user_data.get('first_name', '').strip().title()
13     last_name = user_data.get('last_name', '').strip().title()
14     full_name = f"{first_name} {last_name}"
15     return full_name
16
17
18 def new_name(user_data):
19     street = user_data.get('address', {}).get('street', '').strip()
20     city = user_data.get('address', {}).get('city', '').strip()
21     zip_code = user_data.get('address', {}).get('zip_code', '')
22     formatted_address = f"{street}, {city} {zip_code}"
23     return formatted_address

```

↓

```

1 def process_user_data_before(user_data):
2     full_name = get_full_name(user_data)
3     formatted_address = get_formatted_address(user_data)
4
5     return {
6         'name': full_name,
7         'address': formatted_address
8     }
9
10
11 def get_full_name(user_data):
12     first_name = user_data.get('first_name', '').strip().title()
13     last_name = user_data.get('last_name', '').strip().title()
14     full_name = f"{first_name} {last_name}"
15     return full_name
16
17
18 def get_formatted_address(user_data):
19     street = user_data.get('address', {}).get('street', '').strip()
20     city = user_data.get('address', {}).get('city', '').strip()
21     zip_code = user_data.get('address', {}).get('zip_code', '')
22     formatted_address = f"{street}, {city} {zip_code}"
23     return formatted_address

```

Рис. 10: Пример работы инструмента Extract в случае выделения метода.

Инструмент Inline демонстрирует противоположное по направлению, но не менее важное структурное преобразование. Его задача состоит в том, чтобы устранить промежуточную переменную или другую простую абстракцию и заменить все ее использования исходным вычисляемым выражением. На уровне архитектуры такой инструмент требует особенно аккуратной проверки применимости: необходимо удостовериться, что подстановка не меняет семантику программы, не нарушает порядок вычислений и не приводит к проблемам с побочными эффектами, областями видимости, аугментированными присваиваниями или специальными строковыми конструкциями. Тем самым Inline показывает, что рефакторинг CRC Python IDE умеет не только порождать новую сущность в коде, но и безопасно упрощать уже существующую.

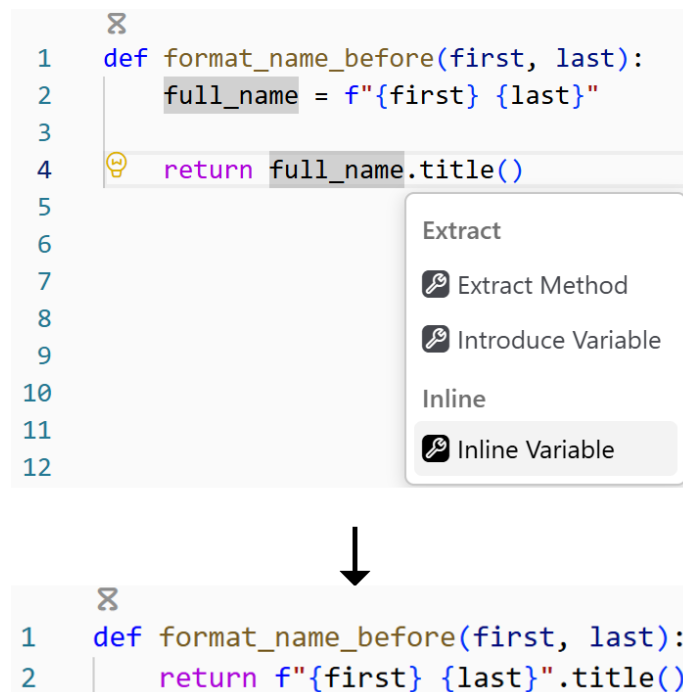


Рис. 11: Пример работы инструмента Inline в случае подстановки переменной.

Инструмент Rename переносит рефакторинг с уровня локального фрагмента кода на уровень семантики всего проекта. В отличие от Extract и Inline, где основное действие сосредоточено вокруг выделенной конструкции, переименование требует согласованной работы с определением идентификатора и всеми его использованиями, потенциально распределенными по нескольким файлам или даже разным зависи-

ностям. Поэтому здесь ключевое значение имеют разрешение целевой сущности, поиск всех ссылок, построение специальных объектов для замены разных типов вхождений и проверка конфликтов именования с учетом их областей видимости. В практическом отношении, Rename является одним из наиболее часто используемых инструментов этой подгруппы.

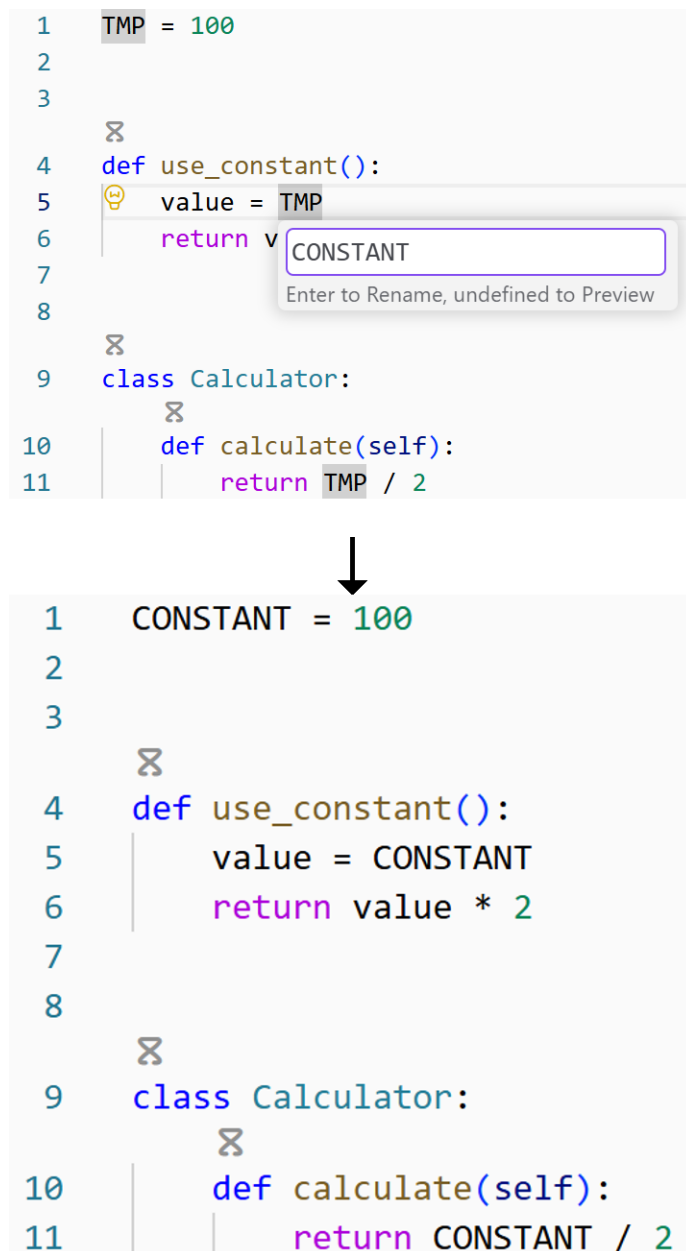


Рис. 12: Пример работы инструмента Rename в случае подстановки переменной.

Инструмент Optimize Imports завершает репрезентативный набор как рефакторинг, работающий уже не с отдельным выражением или

именем, а с семантикой импортов во всем модуле. Его реализация опирается на анализ фактического использования импортированных элементов, удаление неиспользуемых конструкций и последующую нормализацию импортных секций. При этом инструмент учитывает особенности Python-кода, корректно работает с разными видами импортов и согласовывает результат с форматирующей инфраструктурой. Архитектурно он также интересен тем, что находится на стыке нескольких слоев подсистемы: учитывает результаты анализа использований импортов, преобразовывает код и, в то же время, выступает средством нормализации структуры модуля.

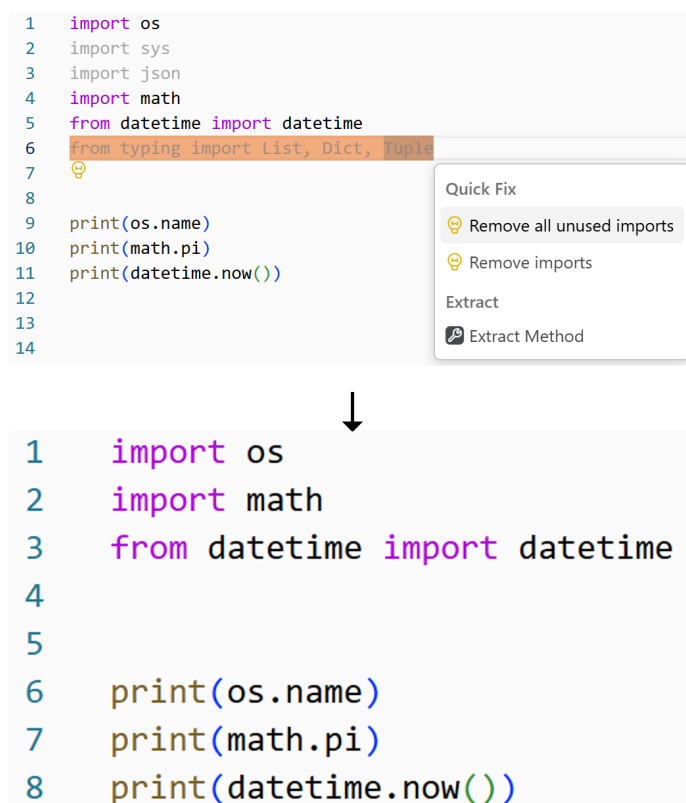


Рис. 13: Пример работы инструмента Optimize Imports в случае удаления неиспользуемых импортов.

Совместно данные инструменты показывают, что подгруппа рефакторинга CRC Python IDE образует самостоятельный и внутренне связанный слой в реализации. Тем самым рефакторинг завершает описание группы инструментов модификации кода и одновременно подводит реализацию к следующей группе возможностей, в которой те же семантические вычисления используются уже не для изменения программы, а

для предоставления пользователю контекстной информации о ней.

5.4. Информационные ассистенты для пользователей

Информационные ассистенты образуют слой пользовательских инструментов для представления семантики программы непосредственно в процессе редактирования кода. В отличие от инструментов статического анализа и модификации кода, эта подгруппа ориентирована на интерпретацию уже имеющегося кода и передачу пользователю сведений, необходимых для понимания текущего контекста. Архитектурно она строится как набор специализированных обработчиков запросов по протоколу LSP, работающих поверх модели кода и выполняющих асинхронную обработку с поддержкой отмены и единообразного преобразования результатов в протокольные структуры данных. Для дальнейшего рассмотрения выбраны инструменты Hover, Signature Help и Inlay Hints, поскольку они покрывают три взаимодополняющих режима ассистирования: точечное обращение к информации, сопровождение ввода и проактивное отображение семантики в самом коде программы.

Инструмент Hover представляет наиболее универсальный сценарий пользовательского информационного ассистирования. Его задача состоит в том, чтобы по позиции курсора выделить семантически значимый элемент программы и сформировать локальное описание, включающее сведения о декларации, типе, документации и, при необходимости, навигационном контексте. Это требует не только работы с идентификаторами, но и интерпретации выражений и некоторых строковых литералов, когда они выступают носителями ссылочной или типовой информации. Дополнительная логика инструмента заключается в том, что он допускает несколько режимов представления результата: при наличии локально вычисляемых сведений они выдаются непосредственно на основе модели кода, а в случае неопределенности могут дополняться средствами генерации документации и типовой информации на основе изучения контекста с помощью ИИ. Поэтому Hover выступает не просто

как всплывающая подсказка, а как комплексная форма пользовательского доступа к семантике программы.

```
1  print()  
2  
3  
4  @overload  
5  def print(  
6      *values: tuple[object],  
7      sep: str | None = " ",  
8      end: str | None = "\n",  
9      file: SupportsWrite[str] | None = None,  
10     flush: Literal[False] = False  
11 ) -> None  
12  
13 Navigate to object overload str SupportsWrite tuple  
14 print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)  
15
```

Рис. 14: Пример работы инструмента Hover в случае наведения на функцию.

Инструмент Signature Help ориентирован на более специализированный, но архитектурно важный сценарий — сопровождение ввода аргументов в точке вызова функции или метода. В отличие от Hover, работающего с произвольной позицией в коде, этот инструмент активируется в контексте списка аргументов и требует точного восстановления структуры вызова. Для этого используются определение положения ввода в списке аргументов, разрешение вызываемого определения функции через анализ перегрузок и вычисление активного параметра по текущему положению каретки. Тем самым Signature Help превращает общую семантическую информацию о вызываемом объекте в короткоживущую, но крайне полезную подсказку, непосредственно сопровождающую процесс написания кода. Его показательность состоит в том, что он демонстрирует способность подсистемы связывать модель кода не только с анализом уже написанной программы, но и с динамикой ее текущего ввода.

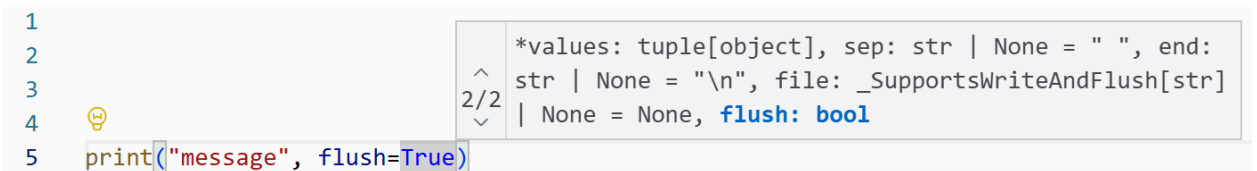


Рис. 15: Пример работы инструмента Signature Help в случае ввода аргументов функции.

Инструмент Inlay Hints представляет третий режим ассистирования — проактивное сопровождение чтения при редактировании программы. Если Hover и Signature Help предоставляют сведения в ответ на действие пользователя, то здесь типовая и иная семантическая информация встраивается непосредственно в текст документа. Инструмент реализован как специализированный обход синтаксического дерева в видимом диапазоне, в ходе которого подсказки вычисляются для присваиваний, параметров функций и возвращаемых типов. Важно отметить, что этот инструмент расположен на границе между информированием пользователя и потенциальной модификацией кода: архитектура допускает применение изменений для вставки типовых аннотаций.

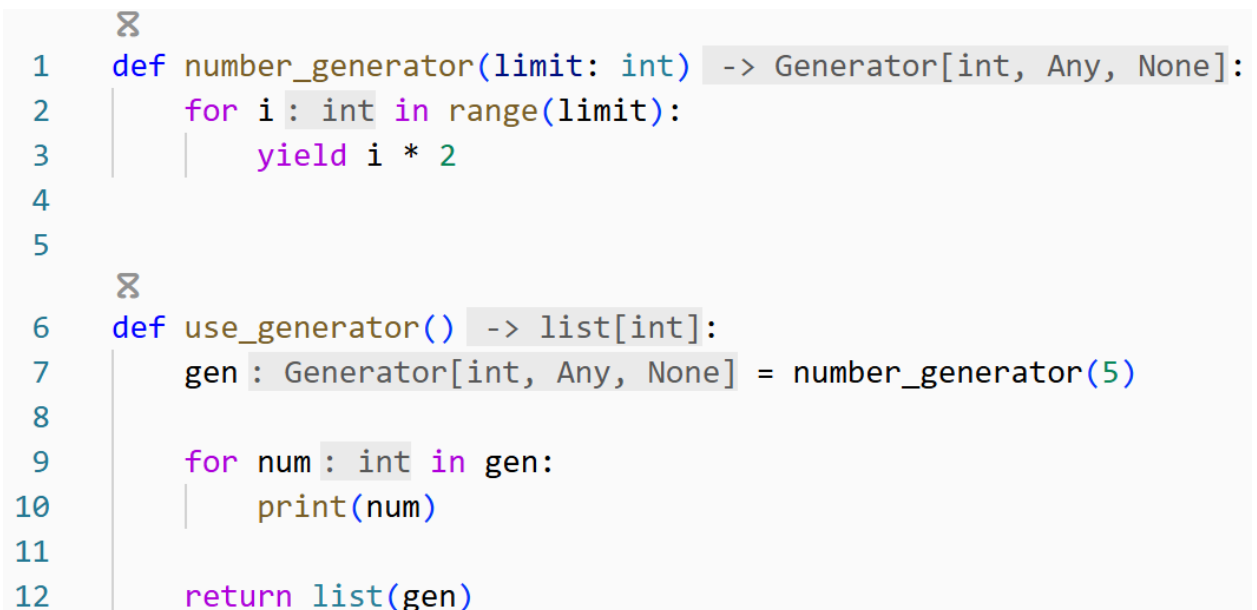


Рис. 16: Пример работы инструмента Inlay Hints в случае отображения типовых аннотаций.

Вместе все эти инструменты демонстрируют три различных способа превращения результатов работы модели кода в инструмент повсе-

дневной разработки. Таким образом, подгруппа информационных ассистентов завершает пользовательский слой подсистемы. Следующий шаг реализации состоит в рассмотрении того, как семантический контекст и безопасные операции над кодом становятся доступными уже не человеку, а внешнему ИИ-агенту через машинно-ориентированный интерфейс.

5.5. Информационные ассистенты для ИИ-агентов

Информационные ассистенты для ИИ-агентов образуют в CRC Python IDE машинно-ориентированный слой доступа к результатам анализа и безопасным операциям над кодом. Их назначение состоит в адаптации уже существующих инструментов к сценарию, в котором потребителем информации выступает внешний ИИ-агент. Архитектурно данная подгруппа является адаптацией инструментов к протоколу MCP. Выполняются прием и валидация параметров запроса, преобразование путей и координат документа, маршрутизация вызова к соответствующему инструменту подсистемы и последующая сериализация результата в унифицированные структуры, пригодные для машинной обработки. Для подробного рассмотрения выбраны инструменты Problems MCP, Hover MCP, Signature Help MCP и Rename MCP, поскольку вместе они покрывают контекстный и операционный режимы агентного взаимодействия с кодом.

Инструмент Problems MCP отвечает за диагностику в данной подгруппе. Он предоставляет агенту структурированный доступ к результатам статического анализа, не создавая отдельной системы проверок, а переиспользуя ту же инфраструктуру, которая используется в пользовательском режиме. По файлу и контексту проекта запускается набор релевантных проверок, после чего результаты фильтруются по диапазону строк и уровню серьезности проблем, а затем переводятся в машинно-читаемый вид. Важное значение имеет именно структура выдачи результата: агент получает не сырой текст о проблеме, а набор записей с диапазонами, типами диагностик и уровнями их серьезности.

Благодаря этому, Problems MСР выступает источником семантического контекста для верификации агентных модификаций кода.

Инструмент Hover MСР адаптирует для ИИ-агента сценарий получения локальной семантической информации о символе в заданной позиции. По своей логике он опирается на тот же механизм, что и пользовательский Hover, однако отличается форматом результата. Визуальные элементы, рассчитанные на человека, здесь отсутствуют, а информация о декларациях, типах и документации преобразуется в структуру, пригодную для дальнейшей обработки агентом. Практическая ценность инструмента состоит в том, что агенту больше не требуется самостоятельно читать файл и восстанавливать ближайший контекст: он получает уже готовые сведения, согласованные с общей моделью кода среды разработки.

Инструмент Signature Help MСР применяет адаптацию к сценарию анализа точки вызова функции, используя существующую логику вычисления сигнатур, параметров и перегрузок, а затем переводит результат в формат, удобный для ИИ-агента. Он получает список параметров текущей сигнатуры, информацию об активном аргументе и список сигнатур всех перегрузок, которых достаточно для выбора корректного способа построения вызова функции. По этой причине Signature Help MСР является показательным примером того, как семантика вызова функции становится доступной ИИ-агенту без дополнительного чтения кода с его стороны.

Инструмент Rename MСР предоставляет ИИ-агенту управляемое действие для переименования элемента. В его основе лежит тот же механизм, который используется и в пользовательских преобразованиях: сначала определяется целевая сущность и область затрагиваемых использований, затем проверяется корректность нового имени и только после этого формируется и применяется набор изменений. Rename MСР делает доступной агенту безопасную операцию над кодом, основанную на модели кода и контроле корректности изменений, что заметно отличает инструмент от стандартной текстовой замены по файлам.

Таким образом, инструменты подгруппы информационных ассистен-

тов для ИИ-агентов являются естественным продолжением уже существующих семантических механизмов, адаптированных под машинно-ориентированный интерфейс.

Теперь реализация инструментов подсистемы CRC Python IDE принимает завершённый вид: статический анализ, модификация кода и информационное ассистирование оказываются доступны как пользователю, так и ИИ-агенту, причем на общей семантической основе в виде модели кода. Теперь необходимо ответить на закономерный вопрос — насколько корректно, устойчиво и практически пригодно ведет себя реализованная подсистема в настоящих сценариях использования.

6. Тестирование и апробация

После описания реализации подсистемы CRC Python IDE необходимо показать, что созданное решение было подтверждено не только на уровне инженерного воплощения, но и на уровне воспроизводимой проверки его корректности, устойчивости и практической пригодности. Для этого используется несколько взаимодополняющих уровней проверки. Модульное тестирование фиксирует локальную корректность отдельных инструментов и общей тестовой инфраструктуры, регрессионное тестирование контролирует устойчивость ключевых характеристик подсистемы на репрезентативных проектах во времени, комплексное тестирование проверяет целые сценарии с участием редактора, языкового сервера, пользовательского интерфейса и внешних сервисов, а апробация по обратной связи от пользователей позволяет оценить практическую ценность системы и уточнить направления ее развития.

6.1. Модульное тестирование

Модульное тестирование подсистемы CRC Python IDE было построено как единая инфраструктура, а не как набор несвязанных модульных тестов для отдельных классов. Ее основу составляет фиксация сценариев, хранимых в виде эталонных файлов, в которых одновременно фиксируются входные данные и ожидаемый выходной результат, который затем сравнивается с фактическим. Такой подход оказался особенно уместным для реализованной подсистемы, поскольку разные подгруппы инструментов имеют разнотипные результаты: статический анализ формирует диагностики, исправления и рефакторинги возвращают модифицированный код, пользовательские ассистенты выдают семантические подсказки, а ассистенты для ИИ-агентов — структурированные ответы в машинно-читаемом формате. Единая тестовая инфраструктура позволила свести все эти неоднородные результаты к воспроизводимой схеме сравнения с эталоном.

Существенным результатом стала модернизация тестовой инфраструктуры. Вместо разрозненных тестовых сценариев была сформиро-

вана единая система с конфигурируемыми тестовыми проектами, общими базовыми абстракциями тестов и унифицированным механизмом сравнения результатов с эталоном. В тестовых данных поддерживаются маркеры позиции курсора и выделенного диапазона, что делает возможной проверку инструментов, зависящих от контекста редактирования. Для каждого сценария фиксируется не только исходный код теста, но и ожидаемое поведение соответствующей группы инструментов, а при расхождении фактического и эталонного результата инфраструктура автоматически фиксирует различия для последующего анализа. Тем самым модульное тестирование подсистемы было превращено в расширяемый инфраструктурный слой, пригодный для покрытия всех возможностей подсистемы.

Отдельную роль играет фиксация ожидаемых результатов для каждой группы инструментов. Для статического анализа эталонный вывод включает диагностические сообщения и их привязку к конкретным позициям в коде, для исправлений и рефакторинга — текст программы до и после применения преобразования, для пользовательских информационных ассистентов — форматированные подсказки или сигнатуры, а для ИИ-агентов — структурированные ответы, пригодные для сравнения в текстовом виде.

Важной особенностью инфраструктуры стала поддержка конфигурируемых тестовых проектов. Тесты выполняются не только над одиночными файлами, но и над проектным окружением с различными режимами, зависимостями и наборами вспомогательных файлов. Это необходимо для тех инструментов, чье поведение зависит от разрешения импортов, доступности внешних зависимостей, режима проекта или версии интерпретатора Python. Поддержка таких сценариев делает модульное тестирование приближенным к настоящему поведению при использовании среды разработки, сохраняя при этом воспроизводимость и расширяемость.

Подготовка инфраструктуры позволила масштабировать тестовое покрытие до нескольких тысяч сценариев. В рамках работы для подсистемы было создано 3000 модульных тестов. Это число отражает си-

систематическое покрытие различных подгрупп инструментов тестовыми сценариями, а также включает их основные, пограничные и сложные случаи. Таким образом, модульное тестирование стало основным средством контроля локальной корректности всей подсистемы.

6.2. Регрессионное тестирование

Если модульное тестирование отвечает прежде всего за воспроизводимую локальную корректность, то регрессионное тестирование направлено на контроль деградаций ключевых характеристик подсистемы во времени. Для CRC Python IDE это особенно важно, поскольку реализованные инструменты опираются на сложные семантические вычисления, применяются в интерактивной среде и должны сохранять приемлемое поведение не только на синтетических примерах, но и на существующих кодовых базах. Поэтому в работе был создан отдельный слой регрессионного тестирования, включающий пять специализированных бенчмарков, по одному для каждой подгруппы инструментов.

Бенчмарки организованы как самостоятельная инфраструктура с единым способом запуска, общими механизмами сбора метрик и поддержкой выполнения как локально, так и в CI/CD-окружении. В них используется 30 выборочных Python-проектов, включающих как известные открытые проекты (Django, Numpy, Transformers и др.), так и внутренние закрытые проекты, предоставленные пользователями. Это принципиально отличает регрессионное тестирование от модульного: здесь проверяется не только правильность результата, но и его стабильность. Благодаря сохранению результатов предыдущих запусков в базе данных Victoria Metrics [23] и отправке метрик в систему мониторинга CRC Python IDE на базе Grafana [4], бенчмарки стали инструментом систематического контроля регрессий и измерения производительности подсистемы.

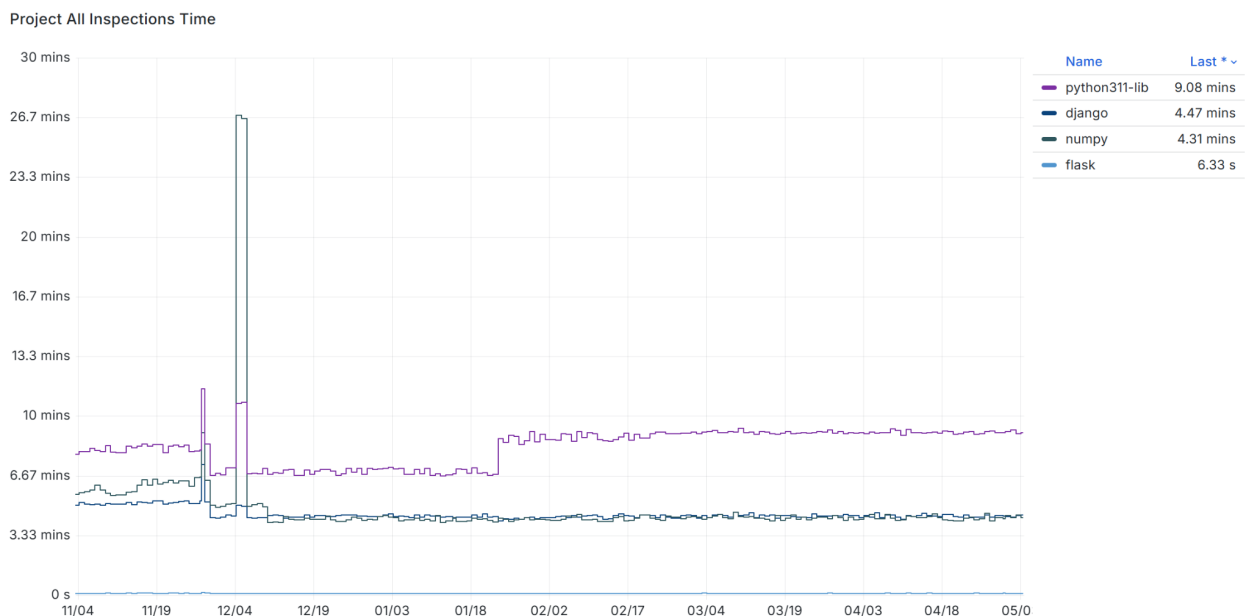


Рис. 17: Визуализация мониторинга подсистемы в Grafana на примере измерения времени полного статического анализа кода некоторых тестовых проектов.

Бенчмарк для инструментов статического анализа ориентирован на наиболее ресурсоемкую часть подсистемы — массовый запуск инспекций на выборочных проектах. В нем измеряются время выполнения анализа, пропускная способность, количество и распределение диагностиков по уровням серьезности, а также показатели точности результатов. Важно на ежедневной основе получать такие результаты, поскольку статический анализ формирует основу для всех последующих подгрупп инструментов: его деградация немедленно отражается и на качестве подсказок, и на корректирующих модификациях, и на рефакторингах.

Бенчмарк для инструментов исправления кода сосредоточен на корректности и скорости применения корректирующих модификаций. В нем проверяется время вычисления и применения исправлений, доля успешно отработавших сценариев и отсутствие роста неверных срабатываний. Данные результаты важны, поскольку даже небольшая регрессия может проявляться либо как ухудшение качества предложений, либо как увеличение задержки пользовательского интерфейса, однако бенчмарк позволяет контролировать оба этих параметра.

Бенчмарк для инструментов рефакторинга проверяет производительность и корректность эквивалентных преобразований кода на слож-

ных проектах. В фокусе находятся контроль количества затрагиваемых файлов, сохранения ссылочной целостности и времени вычисления правок. Эти сведения имеют большое значение, поскольку рефакторинг опирается на поиск использований, разрешение ссылок и безопасное применение множества изменений одновременно. В связи с этим, его регрессионная проверка должна отслеживать не только время выполнения, но и не нарушается ли сама структура преобразуемой программы.

Бенчмарк для информационных ассистентов ориентирован на интерактивные характеристики пользовательского интерфейса: контролируются латентность ответа, полнота предоставляемой информации и устойчивость работы при повторяющихся и параллельных запросах. Здесь критична не только корректность данных, но и время их доставки пользователю, поскольку даже семантически верный результат теряет ценность, если задержка делает инструмент непригодным для повседневного использования. Благодаря этому бенчмарку отслеживаются характеристики, которые определяют пригодность подсказок к интерактивному использованию.

Бенчмарк информационных ассистентов для ИИ-агентов проверяет интеграционный слой между внутренними механизмами и подключаемыми ИИ-агентами. В нем оцениваются время сериализации и десериализации структур данных, корректность формата ответов, полнота передаваемой информации и стабильность работы при серии запросов. Выбор метрик бенчмарка соответствует специфике подгруппы: для агентных инструментов важно не только сохранить семантическую точность, но и сделать ее доступной в машинно-ориентированном формате с приемлемой задержкой.

В совокупности пять представленных бенчмарков покрывают логику работы всех инструментов подсистемы, обеспечивая регрессионный контроль. Их ценность состоит в полном структурном соответствии реализованной подсистеме: деградация фиксируется не абстрактно по всему продукту, а точно по каждой подгруппе инструментов. Это делает регрессионное тестирование самостоятельным и, в то же время, необходимым этапом для подтверждения работоспособности подсистемы, до-

полняя модульное тестирование контролем устойчивости ключевых характеристик во времени.

6.3. Комплексное тестирование

Комплексное тестирование предназначено для проверки тех свойств подсистемы, которые проявляются только при совместной работе редактора, языкового сервера, пользовательского интерфейса и внешних сервисов. Для CRC Python IDE такой уровень проверки особенно важен, поскольку значительная часть реализованных инструментов взаимодействуют между собой в среде разработки, а потому значение имеют асинхронность, время доставки результата до пользовательского интерфейса, корректность навигации, работа диалогов и согласованность нескольких компонентов сразу. Поэтому в качестве комплексного тестирования были настроены автотесты на основе фреймворка Selenium [20].

Инфраструктура этих тестов построена по принципу Page Object Model (ПОМ) и использует отдельный слой компонент, пользовательских действий и сценариев проверки. Это позволяет тестировать решение не как набор прямых обращений к элементам интерфейса, а как систему воспроизводимых пользовательских сценариев: запуск среды, открытие проекта, работа с редактором, ожидание появления результатов анализа, вызов действий над кодом, применение изменений и проверка состояния интерфейса после завершения операций. Существенную роль здесь играют механизмы синхронизации и ожидания, поскольку взаимодействие между редактором и языковым сервером носит асинхронный характер, а часть сценариев дополнительно зависит от внешних сервисов.

Для инструментов статического анализа комплексные тесты покрывают полный путь от открытия или изменения файла до появления диагностик в интерфейсе, их корректной классификации, фильтрации и навигации к месту проблемы. Здесь проверяется уже не только внутренняя логика статического анализа, но и как его результаты доходят

до пользователя через интерфейс. Такая проверка позволяет ловить ошибки синхронизации, отображения и отправки диагностик, которые не проявляются в модульных тестах, где проверяется лишь сам вычисленный результат.

Для инструментов исправления кода сценарии комплексных тестов сосредоточены на получении и применении корректирующих модификаций через интерфейс среды разработки. Тесты проверяют появление доступных действий по диагностике, корректность их отображения, применение правок и последующее обновление состояния редактора и списка проблем. Именно на этом уровне можно обнаружить ошибки, связанные с согласованностью анализа и исправления кода, потерей фокуса, неправильным обновлением документа или несвоевременным исчезновением проблемы из списка.

Для инструментов рефакторинга кода комплексное тестирование покрывает сценарии, в которых пользователь инициирует преобразование через интерфейс, просматривает предполагаемые изменения и применяет их к проекту. Особое значение здесь имеет проверка многошаговых операций, например, переименования с обновлением всех использований и предпросмотром результатов для подтверждения. Такой уровень тестирования позволяет убедиться, что механизм рефакторинга действительно работает как целостный сценарий.

Пользовательские информационные ассистенты представлены в комплексном тестировании сценариями получения подсказок и других контекстных сведений непосредственно в редакторе. Здесь проверяется то, как семантические данные появляются в интерфейсе, с какой задержкой, в каком виде и насколько устойчиво ведут себя при повторных запросах и перемещении курсора. Тем самым комплексное тестирование фиксирует качество доставки информации пользователю, что особенно важно для интерактивных ассистентов сопровождения кода.

Для информационных ассистентов, связанных с ИИ-агентами, комплексные сценарии сосредоточены на интеграции с внешними сервисами: открытии интерфейса чата, передаче туда контекста с кодом, получении ответа и его отображении в пользовательском интерфейсе. Это

распространяется на сценарии объяснения кода, генерации кода и интерактивных подсказок. Такие тесты позволяют выявлять проблемы на стыке клиента, языкового сервера и внешнего сервиса, где важно воспроизвести задержки, таймауты и особенности асинхронного взаимодействия, приближенные к настоящим.

Таким образом, комплексное тестирование охватывает именно те интеграционные риски, которые не могут быть полностью выявлены ни модульными тестами, ни регрессионными бенчмарками: ошибки синхронизации между компонентами, сбои пользовательского интерфейса, некорректную доставку результатов анализа до пользователя и проблемы с применением модификаций кода.

6.4. Апробация

Апробация подсистемы направлена на проверку ее практической полезности в условиях использования CRC Python IDE и на выявление тех сценариев, которые невозможно полностью предусмотреть подготовленными тестами. На начальных этапах разработки основными пользователями новых инструментов выступали сами разработчики CRC Python IDE, что позволяло оперативно проверять подсистему в повседневных рабочих сценариях. По мере стабилизации решения, круг пользователей расширился и включил сторонних пользователей внутри компании, благодаря чему обратная связь стала более разнообразной и позволила оценить поведение инструментов за пределами собственного окружения разработки.

Отзывы, полученные в ходе использования, оказались полезны не только как подтверждение востребованности уже реализованных возможностей, но и как источник дальнейшего развития подсистемы. По результатам апробации были добавлены три новых инструмента. Инструмент Version Compatibility расширил группу статического анализа проверкой совместимости используемых языковых возможностей Python с выбранной версией интерпретатора и, тем самым, закрыл практический запрос на раннее выявление несовместимых конструк-

ций. Инструмент Add Superclass Constructor Call дополнил подгруппу исправлений кода возможностью автоматически добавлять вызов конструктора суперкласса с передачей подходящих аргументов, что облегчило работу со сценариями, связанными с наследованием и инициализацией объектов. Инструмент Optimize Imports усилил подгруппу рефакторинга за счет автоматической очистки и упорядочивания импортов, объединяя анализ фактического использования конструкций с форматированием структуры модуля.

Не менее важным результатом апробации стало исправление десяти проблем, выявленных при практическом использовании подсистемы. Эти проблемы относились к разным аспектам ее работы и поэтому хорошо отражают сложность разработанного решения. По обратной связи пользователей были устранены ложно-положительные и ложно-отрицательные срабатывания в статическом анализе, скорректированы некорректные или неэквивалентные модификации кода, а также исправлены нерелевантные подсказки и другие сценарии пользовательского взаимодействия с инструментами. Так, апробация решения стала еще одним способом уточнения логики инструментов.

С инженерной точки зрения, апробация полезна тем, что подтверждает способность подсистемы к развитию без пересмотра ее фундаментальной архитектуры. Новые инструменты были встроены в уже существующие группы средств анализа и модификации кода, а исправление выявленных проблем осуществлялось в логике ранее спроектированных механизмов. Потому, обратная связь от пользователей не стала источником хаотических доработок, а, наоборот, показала, что архитектурные решения действительно допускают расширение, уточнение и повышение качества работы подсистемы. Это позволяет перейти к итоговому обобщению результатов работы.

7. Заключение

Для достижения цели выпускной квалификационной работы были получены следующие результаты:

- Рассмотрены инструменты для анализа и ассистирования модификаций кода в статических анализаторах Муру, Pyright и Pyrefly, а также в средах разработки (IDE) PyCharm, VSCode и CRC Python IDE и, наконец, в системах на основе ИИ-агентов (ADE) AI Assistant, Copilot и Cursor. Определены ключевые особенности этих решений и соответствующие им запросы Python-разработчиков.
- Спроектирована подсистема среды разработки, включающая 3 следующие группы инструментов: (i) статический анализ, (ii) исправление и рефакторинг кода, а также (iii) информационные ассистенты для пользователей и ИИ-агентов. Спроектированы необходимые улучшения модели кода CRC Python IDE: расширение и стабилизация синтаксического и семантического представления программы (PSI), уточнение и конфигурирование возможностей по разрешению ссылок (Resolve), доработка интерфейса вывода типов (Type Inference).
- Выполнена реализация спроектированной подсистемы:
 - реализованы улучшения в модели кода CRC Python IDE;
 - реализовано 40 инструментов для статического анализа кода (Syntax Correctness, Reference Resolve, Expected Type, Unreachable Code и др.);
 - реализовано 12 инструментов для исправления кода (Install Package, Import Reference, Create Reference, Add Qualifier и др.);
 - реализовано 8 инструментов для рефакторинга кода (Extract, Inline, Rename, Optimize Imports и др.);

- реализовано 3 информационных ассистента для пользователей (Hover, Signature Help и Inlay Hints);
 - реализовано 4 информационных ассистента для ИИ-агентов (Problems MCP, Hover MCP, Signature Help MCP и Rename MCP).
- Выполнена модернизация тестовой инфраструктуры CRC Python IDE (добавлена поддержка конфигурируемых тестовых проектов с фиксацией ожидаемых результатов для каждой группы инструментов), создано 3000 модульных тестов и 5 бенчмарков для регрессионного тестирования подсистемы, а в качестве комплексного тестирования настроены автотесты с помощью фреймворка Selenium.
 - Проведена апробация подсистемы на основе отзывов разработчиков и пользователей CRC Python IDE, что способствовало созданию 3 новых инструментов (Version Compatibility, Add Superclass Constructor Call и Optimize Imports) и исправлению 10 проблем (ложно-положительные/отрицательные срабатывания и др.).

Список литературы

- [1] Agarwal V., Pei Y., Alamir S., Liu X. CodeMirage: Hallucinations in Code Generated by Large Language Models. — 2024. — URL: <https://doi.org/10.48550/arXiv.2408.08333> (дата обращения: 6 мая 2026 г.).
- [2] Cursor. Cursor Documentation. — URL: <https://cursor.com/en-US/docs> (дата обращения: 6 мая 2026 г.).
- [3] GitHub. GitHub Copilot Documentation. — URL: <https://docs.github.com/en/copilot> (дата обращения: 6 мая 2026 г.).
- [4] Grafana Labs. Grafana Documentation. — URL: <https://grafana.com/docs/> (дата обращения: 6 мая 2026 г.).
- [5] Huawei Cloud. CRC Python IDE Documentation. — 2025. — URL: https://support.huaweicloud.com/usermanual-codeartside/codeartside_01_0361.html (дата обращения: 6 мая 2026 г.).
- [6] Jaoua I., Ben Sghaier O., Sahraoui H. Combining Large Language Models with Static Analyzers for Code Review Generation // IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR). — 2025. — URL: <https://doi.org/10.48550/arXiv.2502.06633> (дата обращения: 6 мая 2026 г.).
- [7] JetBrains. AI Assistant Documentation. — URL: <https://www.jetbrains.com/help/ai-assistant> (дата обращения: 6 мая 2026 г.).
- [8] JetBrains. PyCharm Documentation. — URL: <https://www.jetbrains.com/help/pycharm> (дата обращения: 6 мая 2026 г.).
- [9] JetBrains. Python Developers Survey 2024. — 2024. — URL: <https://lp.jetbrains.com/python-developers-survey-2024/> (дата обращения: 6 мая 2026 г.).

- [10] Wang C., Huang K., Zhang J. et al. LLMs Meet Library Evolution: Evaluating Deprecated API Usage in LLM-based Code Completion. — 2024. — URL: <https://doi.org/10.48550/arXiv.2406.09834> (дата обращения: 6 мая 2026 г.).
- [11] Large Language Models for Software Engineering: Systematic Literature Review / X. Hou, Y. Zhao, Y. Liu et al. // [ACM Transactions on Software Engineering and Methodology](https://dl.acm.org/doi/10.1145/3695988). — 2024. — URL: <https://dl.acm.org/doi/10.1145/3695988> (дата обращения: 6 мая 2026 г.).
- [12] Microsoft. Language Server Protocol Specification. — URL: <https://microsoft.github.io/language-server-protocol> (дата обращения: 6 мая 2026 г.).
- [13] Microsoft. Pylance Documentation. — URL: <https://github.com/microsoft/pylance-release> (дата обращения: 6 мая 2026 г.).
- [14] Microsoft. Pyright Documentation. — URL: <https://microsoft.github.io/pyright> (дата обращения: 6 мая 2026 г.).
- [15] Microsoft. Visual Studio Code Python Documentation. — URL: <https://code.visualstudio.com/docs/python> (дата обращения: 6 мая 2026 г.).
- [16] Model Context Protocol. Model Context Protocol Documentation. — URL: <https://modelcontextprotocol.io/docs> (дата обращения: 6 мая 2026 г.).
- [17] Мypy. Мypy Documentation. — URL: <https://mypy.readthedocs.io> (дата обращения: 6 мая 2026 г.).
- [18] Pyrefly. Pyrefly Documentation. — URL: <https://pyrefly.org/en/docs> (дата обращения: 6 мая 2026 г.). Документация проекта Пурегу.
- [19] Python Software Foundation. PEP 484 – Type Hints. — 2014. — URL: <https://peps.python.org/pep-0484> (дата обращения: 6 мая 2026 г.).

- [20] Selenium. Selenium Documentation. — URL: <https://www.selenium.dev/documentation/> (дата обращения: 6 мая 2026 г.).
- [21] Sergeyuk A., Titov S., Izadi M. [In-IDE Human-AI Experience in the Era of Large Language Models: A Literature Review](#) // Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments. — 2024. — URL: <https://dl.acm.org/doi/10.1145/3643796.3648463> (дата обращения: 6 мая 2026 г.).
- [22] Stack Overflow. Stack Overflow Developer Survey 2025. — 2025. — URL: <https://survey.stackoverflow.co/2025/> (дата обращения: 6 мая 2026 г.).
- [23] VictoriaMetrics. VictoriaMetrics Documentation. — URL: <https://docs.victoriametrics.com/> (дата обращения: 6 мая 2026 г.).
- [24] Божнюк А. Система модификации структуры исходного кода для интегрированных сред разработки. — 2023. — СПбГУ.
- [25] Мирошников В. Статический вывод типов для языка Python в интегрированной среде разработки. — 2023. — СПбГУ.
- [26] Мирошников В. Инкрементальный статический вывод типов для языка Python. — 2025. — СПбГУ.
- [27] Степанов С. Потокочувствительный анализ программ на языке Python. — 2023. — ИТМО.

А. Приложение

В приложении представлен полный сгруппированный список инструментов, реализованных в рамках подсистемы CRC Python IDE. Для каждого инструмента приводится его название, краткое описание и особенности, а также технологии, на которых он основан.

А.1. Инструменты для статического анализа кода

Таблица 4 – Инструменты для статического анализа кода

№	Название	Описание	Предлагаемые модификации	Используемые технологии
1.	Abstract Class Instantiated	Обнаружение создания объекта абстрактного класса или класса с нереализованными абстрактными методами	Remove Call, Implement Methods	PSI, Resolve, Type Inference
2.	Operator Usage	Проверка реализации специальных методов у типов при использовании унарных, бинарных и присваивающих операторов	Remove Operator, Implement Methods	PSI, Resolve, Type Inference
3.	Call Usage	Проверка реализации специальных методов у типов при использовании вызова	Remove Call, Implement Methods	PSI, Resolve, Type Inference
4.	Iteration Usage	Проверка реализации специальных методов у типов при использовании итерирования	Remove For, Implement Methods	PSI, Resolve, Type Inference
5.	Subscription Usage	Проверка реализации специальных методов у типов при обращении через индекс	Remove Subscription, Implement Methods	PSI, Resolve, Type Inference

Продолжение таблицы 4

№	Название	Описание	Предлагаемые модификации	Используемые технологии
6.	Deprecated Usage	Проверка использования операторов, классов, функций, методов и модулей, помеченных как устаревшие		PSI, Resolve, Type Inference
7.	Decorator Usage	Проверка использования декораторов для классов, функций и методов	Remove Decorator	PSI, Resolve, Type Inference
8.	Specific Decorator Usage	Проверка использования известных декораторов, обладающих специальной семантикой	Remove Decorator	PSI, Resolve, Type Inference
9.	Superclass Duplicate	Обнаружение дубликатов суперклассов в иерархии наследования	Remove Superclass	PSI, Resolve, Type Inference
10.	Dictionary Keys Duplicate	Обнаружение дубликатов ключей в словарях	Remove Dictionary Key	PSI, Resolve, Type Inference
11.	Except Exception Inheritance	Проверка исключения в конструкции “try-except” на наличие класса Exception в иерархии наследования	Add Exception Superclass	PSI, Resolve, Type Inference
12.	Raise Exception Inheritance	Проверка исключения в конструкции “raise” на наличие класса Exception в иерархии наследования	Add Exception Superclass	PSI, Resolve, Type Inference
13.	Name Shadowing	Анализ идентификаторов импортов, классов, функций, методов, полей, переменных и параметров на сокрытие имен из внешнего контекста	Rename, Exclude Name From Shadowing	PSI, Resolve, SSA, Type Inference

Продолжение таблицы 4

№	Название	Описание	Предлагаемые модификации	Используемые технологии
14.	Name Redeclaration	Анализ идентификаторов импортов, классов, функций, методов, полей, переменных и параметров на лишние пере-присваивания	Rename, Exclude Name From Redeclaration	PSI, Resolve, SSA, Type Inference
15.	Unreachable Code	Обнаружение недостижимого кода	Remove Unreachable Code	PSI, Resolve, SSA, Type Inference
16.	Reference Resolve	Анализ разрешимости импортов, простых и квалифицированных ссылок	Import Reference, Install Package, Add Self / Cls / Super Qualifier, Create Class / Function / Method / Field / Variable / Parameter	PSI, Resolve, SSA, Type Inference
17.	Reference Usage	Обнаружение неиспользуемых ссылок у импортов, классов, функций, методов, полей, переменных и параметров	Optimize Imports, Remove Import / Class / Function / Method / Field / Variable / Parameter	PSI, Resolve, SSA, Type Inference
18.	Argument List	Анализ корректности аргументов на основе параметров: сопоставление типов, проверка порядка, значений, обнаружение пропущенных, повторяющихся и лишних аргументов	Rename, Remove Argument	PSI, Resolve, Type Inference
19.	Parameter List	Анализ корректности параметров: проверка порядка, значений и специальных параметров методов	Rename Self / Cls Parameter, Create Self / Cls Parameter	PSI, Resolve, Type Inference

Продолжение таблицы 4

№	Название	Описание	Предлагаемые модификации	Используемые технологии
20.	Target Assignment	Анализ возможности присваивания в выражение	Remove Assignment, Implement Methods	PSI, Resolve, Type Inference
21.	None Assignment	Анализ присваиваний результатов функций и методов, возвращающих None	Remove Assignment	PSI, Resolve, Type Inference
22.	None Comparison	Анализ корректности операторов сравнения с None	Replace Equality	PSI
23.	Missing Async Modifier	Обнаружение конструкций “await”, “async for” и “async with” вне контекста асинхронной функции или метода	Remove Async Modifier, Add Async Modifier	PSI
24.	Expected Type	Сопоставление выведенного и ожидаемого типа для выражений	Convert Annotation To Optional	PSI, Resolve, Type Inference
25.	If Simplification	Обнаружение эквивалентных упрощений в конструкциях “if-else”	Simplify If / Min / Max	PSI
26.	Comparison Simplification	Обнаружение эквивалентных упрощений для конструкций сравнения	Simplify Negative Comparison	PSI
27.	Misplaced Default Except	Обнаружение некорректной позиции обработчика исключений по умолчанию в конструкциях “try-except”	Move Except, Remove Except	PSI
28.	Missing Reraised Exception	Проверка, что в контексте есть исключение, которое может быть делегировано	Remove Raise	PSI

Продолжение таблицы 4

№	Название	Описание	Предлагаемые модификации	Используемые технологии
29.	Conflicting Except And Star Except	Обнаружение одновременно-го использования конструкций “except” и “except*” в конструкции “try-except”	Remove Except	PSI
30.	Missing Abstract Class	Обнаружение абстрактных методов вне контекста абстрактного класса	Add ABC Superclass, Add ABCMeta Metaclass, Implement Methods	PSI, Resolve, Type Inference
31.	Missing Superclass Constructor Call	Анализ конструкторов на необходимость вызова конструктора суперкласса	Add Superclass Constructor Call	PSI, Resolve, Type Inference
32.	Misplaced Return	Проверка, что “return” находится внутри функции или метода	Remove Return	PSI
33.	Misplaced Yield	Проверка, что “yield” находится внутри функции или метода	Remove Yield	PSI
34.	Misplaced Star Import	Проверка, что конструкция “import *” находится на уровне модуля	Remove Import, Move Import	PSI
35.	Missing Loop	Проверка, что “break” и “continue” находятся внутри цикла	Remove Break, Remove Continue	PSI
36.	Syntax Correctness	Анализ синтаксической корректности кода		PSI
37.	Incorrect Formatting	Анализ форматирования и стиля кода	Reformat	PSI
38.	Type Annotation	Проверка корректности типовых аннотаций	Remove Type Annotation	PSI

Продолжение таблицы 4

№	Название	Описание	Предлагаемые модификации	Используемые технологии
39.	Trailing Semicolon	Обнаружение лишних точек с запятой	Remove Semicolon	PSI
40.	Version Compatibility	Анализ совместимости используемых языковых возможностей Python и выбранной версии интерпретатора	Rename, Remove	PSI, Resolve, Type Inference

А.2. Инструменты для исправления кода

Таблица 5 – Инструменты для исправления кода

№	Название	Описание	Используемые технологии
1.	Install Package	Установка выбранного пакета из известных источников с помощью менеджера пакетов	PSI, Package Manager
2.	Import Reference	Добавление выбранного импорта для ссылки на уровень модуля или локально	PSI, WritePSI, Resolve, Type Inference
3.	Create Reference	Создание класса, функции, метода, поля, переменной, параметра для идентификатора с учетом контекста	PSI, WritePSI, Resolve, Type Inference
4.	Add Qualifier	Добавление выбранного квалификатора к идентификатору	PSI, WritePSI
5.	Add Superclass	Добавление суперкласса в сигнатуру класса	PSI, WritePSI
6.	Add Metaclass	Добавление метакласса в сигнатуру класса	PSI, WritePSI
7.	Add Superclass Constructor Call	Добавление вызова конструктора суперкласса с передачей подходящих аргументов	PSI, WritePSI, Resolve, Type Inference

Продолжение таблицы 5

№	Название	Описание	Используемые технологии
8.	Add Async Modifier	Добавление модификатора “async” для внешней конструкции	PSI, WritePSI
9.	Convert Annotation To Optional	Преобразование аннотации к типу со значением None	PSI, WritePSI
10.	Simplify If, Min, Max, Negative Comparison	Эквивалентное упрощение или замена на вызов функции в конструкциях “if-else” и при сравнении с отрицанием	PSI, WritePSI
11.	Replace Equality	Замена операторов сравнения с None на конструкции “is” и “is not”	PSI, WritePSI
12.	Exclude Name	Добавление идентификатора в список исключений с помощью конфигураций	PSI, Configuration

А.3. Инструменты для рефакторинга кода

Таблица 6 – Инструменты для рефакторинга кода

№	Название	Описание	Используемые технологии
1.	Extract	Выделение переменных, полей и функций из выбранного кода	PSI, WritePSI, Resolve, SSA, Type Inference
2.	Inline	Подстановка выбранных переменных, полей и функций в код	PSI, WritePSI, Resolve, SSA, Type Inference
3.	Move	Перемещение выбранных конструкций в коде	PSI, WritePSI
4.	Remove	Удаление выбранных конструкций в коде	PSI, WritePSI

Продолжение таблицы 6

№	Название	Описание	Используемые технологии
5.	Rename	Переименование выбранного идентификатора	PSI, WritePSI, Resolve, SSA, Type Inference
6.	Reformat	Форматирование кода с помощью выбранного инструмента для форматирования и стиля кода	PSI, WritePSI, Formatter
7.	Optimize Imports	Оптимизация импортов путем удаления неиспользуемых конструкций и сортировки с помощью выбранного инструмента для форматирования и стиля кода	PSI, WritePSI, Resolve, SSA, Type Inference, Formatter
8.	Implement Methods	Реализация или переопределение метода	PSI, WritePSI, Resolve, SSA, Type Inference

А.4. Информационные ассистенты для пользователей

Таблица 7 – Информационные ассистенты для пользователей

№	Название	Описание	Используемые технологии
1.	Hover	Подсказка информации о декларациях, типах и документации при наведении на идентификатор в коде	PSI, Resolve, Type Inference, RAG, LLM

Продолжение таблицы 7

№	Название	Описание	Используемые технологии
2.	Signature Help	Подсказка информации о сигнатурах, типах и документации при передаче аргументов в вызов функции	PSI, Resolve, Type Inference, RAG, LLM
3.	Inlay Hints	Подсказка информации об именах и типах идентификаторов с возможностью вставки в код	PSI, WritePSI, Resolve, Type Inference

А.5. Информационные ассистенты для ИИ-агентов

Таблица 8 – Информационные ассистенты для ИИ-агентов

№	Название	Описание	Используемые технологии
1.	Problems MCP	Получение ИИ-агентом результатов статического анализа кода для повышения качества модификаций кода без дополнительного использования инструментов чтения кода	Static Analysis, Chat, MCP, LLM
2.	Hover MCP	Получение ИИ-агентом информации о декларациях, типах и документации идентификатора без дополнительного использования инструментов чтения кода	Hover, Chat, MCP, LLM
3.	Signature Help MCP	Получение ИИ-агентом информации о сигнатурах, типах и документации вызова функции без дополнительного использования инструментов чтения кода	Signature Help, Chat, MCP, LLM
4.	Rename MCP	Переименование ИИ-агентом идентификатора без дополнительного использования инструментов чтения кода	Rename, Chat, MCP, LLM